EXP NO: 1	AZURE DEVOPS ENVIRONMENT SETUP

Aim:

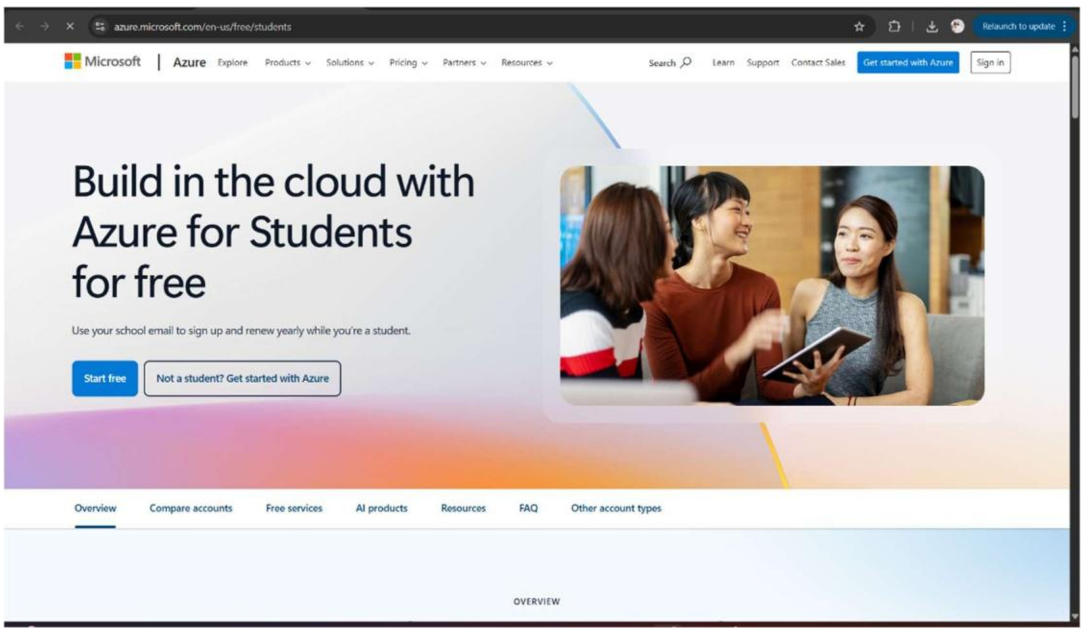
To set up and access the Azure DevOps environment by creating an organization through the Azure portal.

INSTALLATION

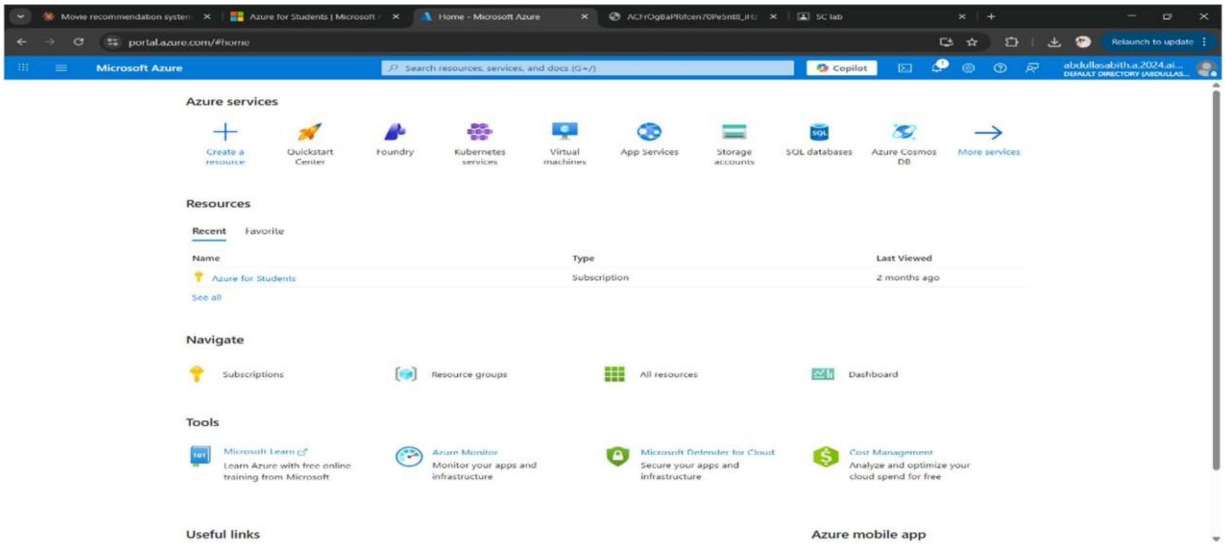
1.Open your web browser and go to the Azure website: https://azure.microsoft.com/en-us/getstarted/azureportal.

Sign in using your Microsoft account credentials.

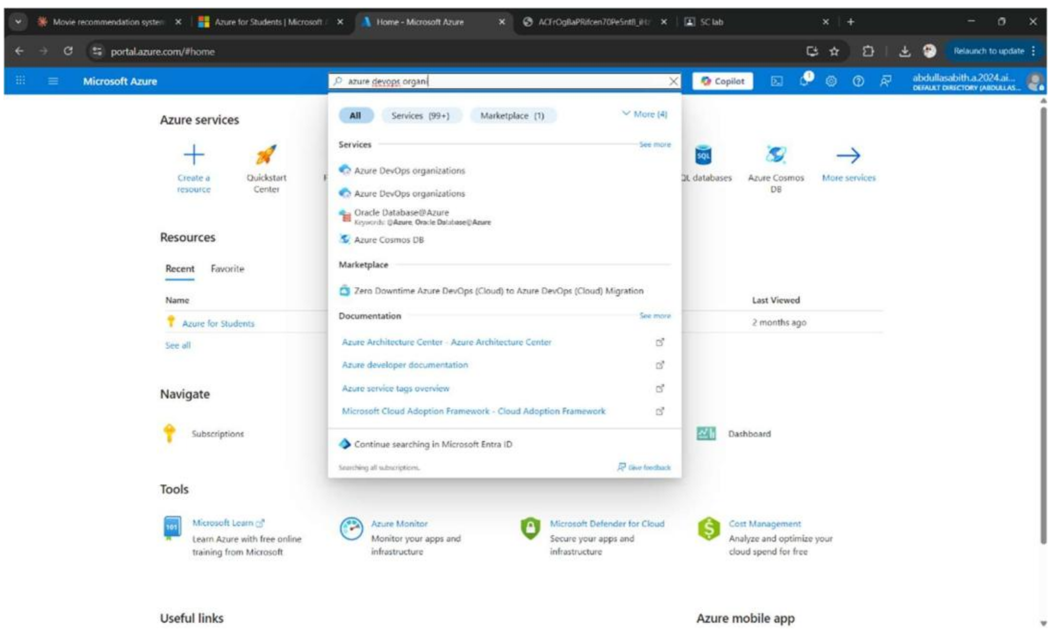
If you don't have a Microsoft account, you can create one here: https://signup.live.com/?lic=1



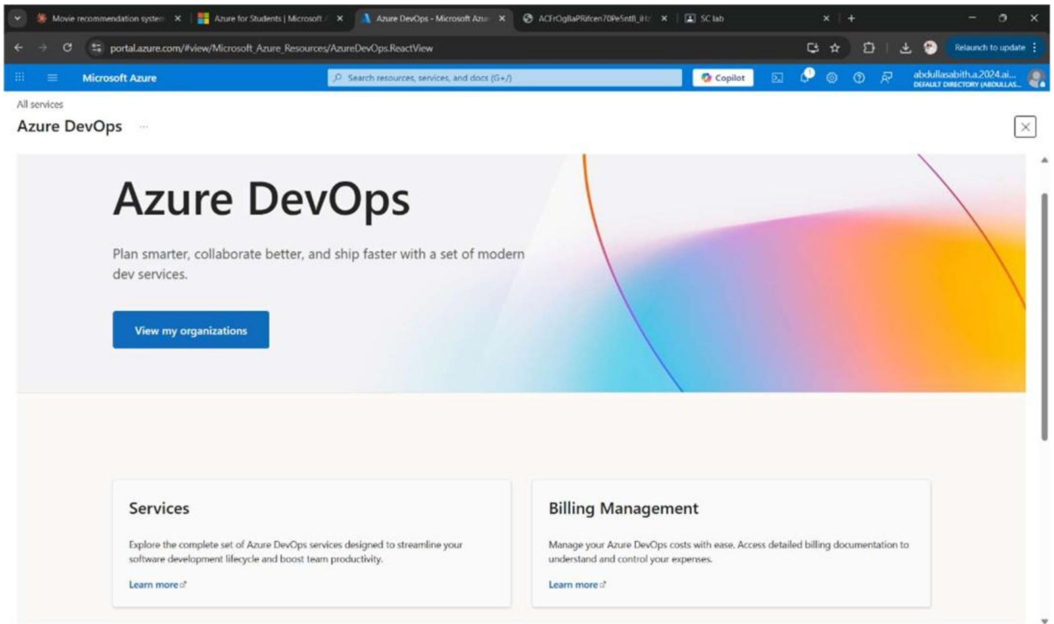
2.Azure home page



3.Open DevOps environment in the Azure platform by typing Azure DevOps Organizations in the search bar.



4.Click on the ***My Azure DevOps Organization*** link and create an organization and you should be taken to the Azure DevOps Organization Home page.



Result:

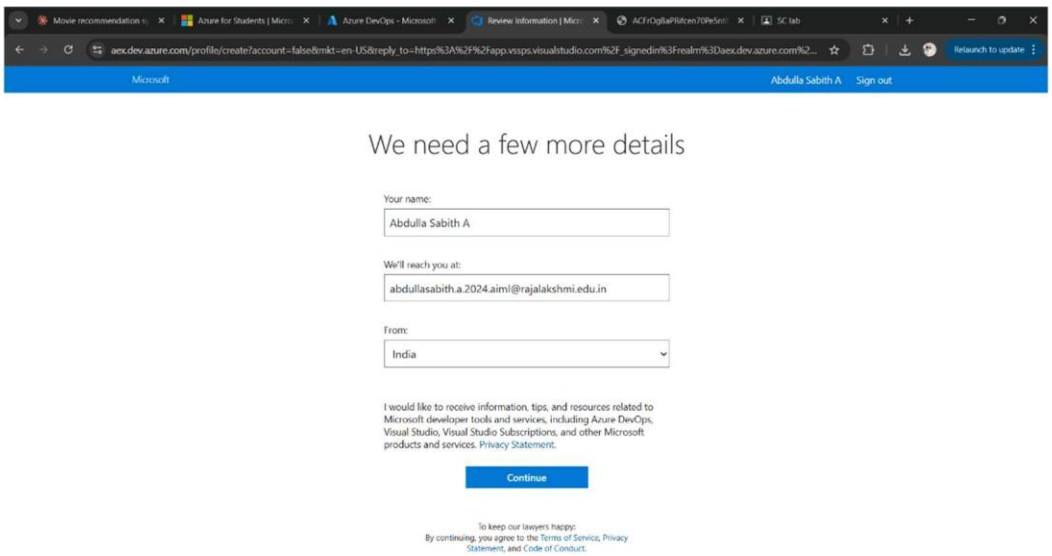
Successfully accessed the Azure DevOps environment and created a new organization through the Azure portal.

EXP NO: 2	**AZURE DEVOPS PROJECT SETUP AND USER STORY MANAGEMENT**

Aim:

To set up an Azure DevOps project for efficient collaboration and agile work management.

1.Create An Azure Account



2.Create the First Project in Your Organization

a. After the organization is set up, you'll need to create your first **project**. This is where you'll begin to manage code, pipelines, work items, and more.

b. On the organization's **Home page**, click on the **New Project** button.

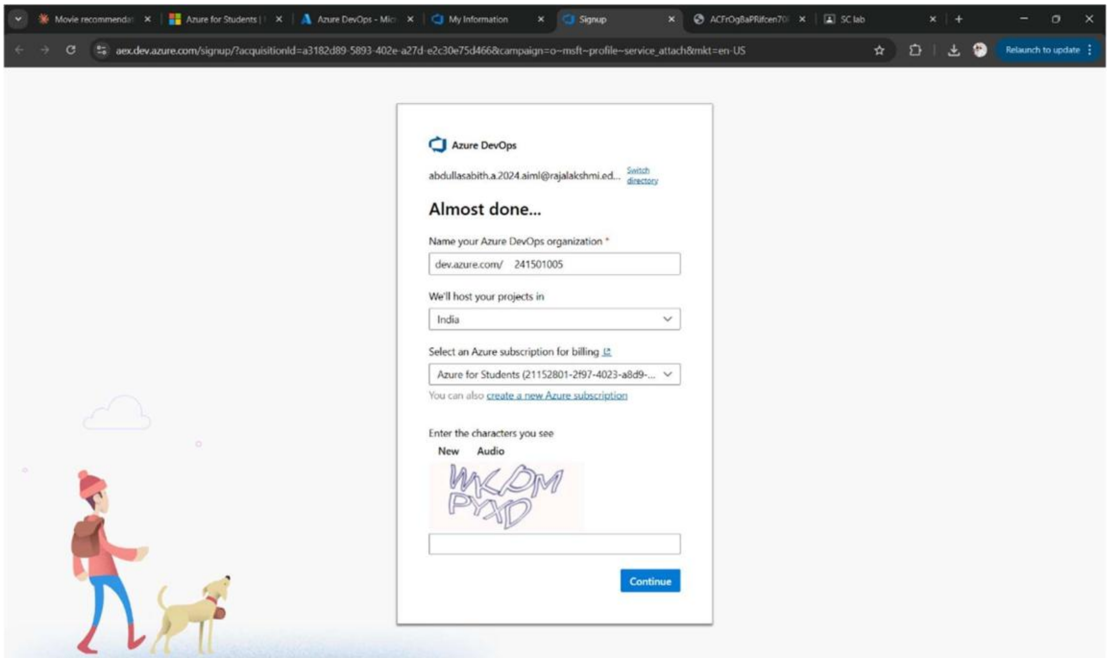
c. Enter the project name, description, and visibility options:

Name: Choose a name for the project (e.g., LMS).

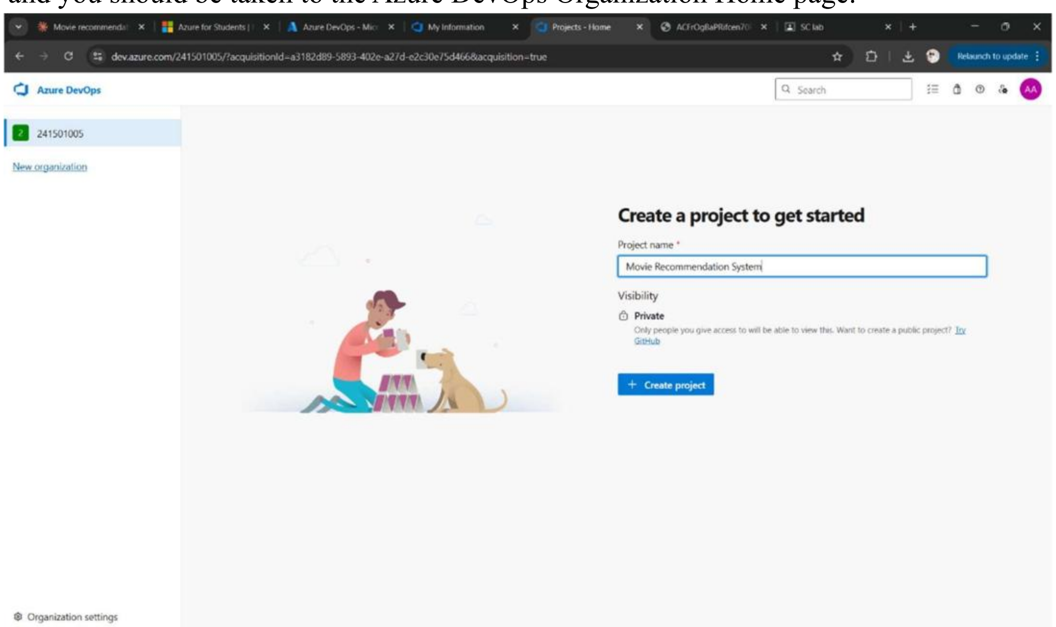
Description: Optionally, add a description to provide more context about the project.

Visibility: Choose whether you want the project to be **Private** (accessible only to those invited) or **Public** (accessible to anyone).

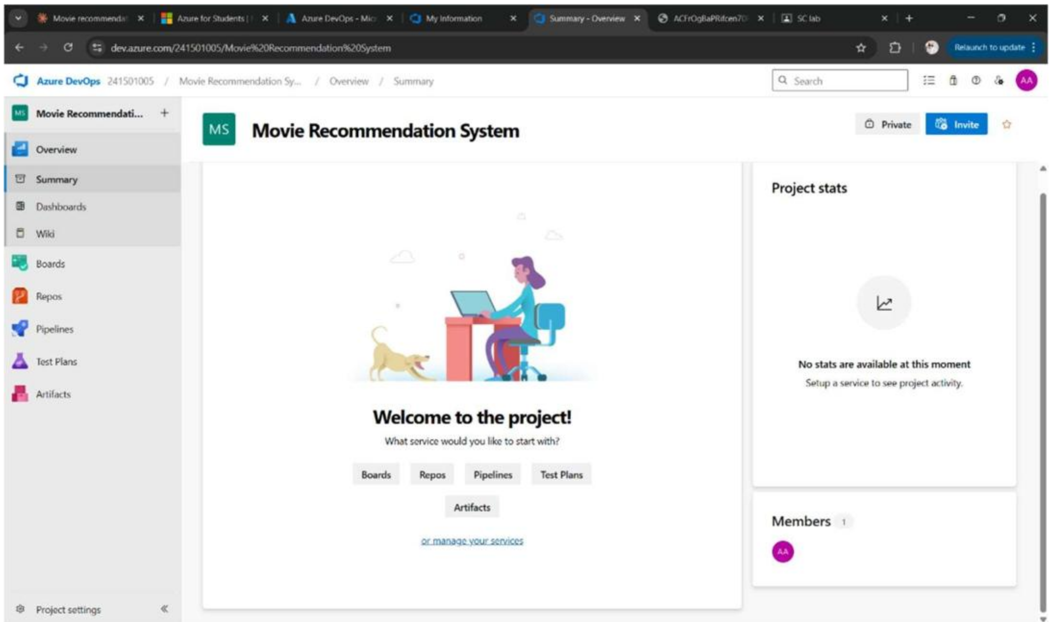
d. Once you've filled out the details, click **Create** to set up your first project.



3.Once logged in, ensure you are in the correct organization. If you're part of multiple organizations, you can switch between them from the top left corner (next to your user profile). Click on the Organization name, and you should be taken to the Azure DevOps Organization Home page.



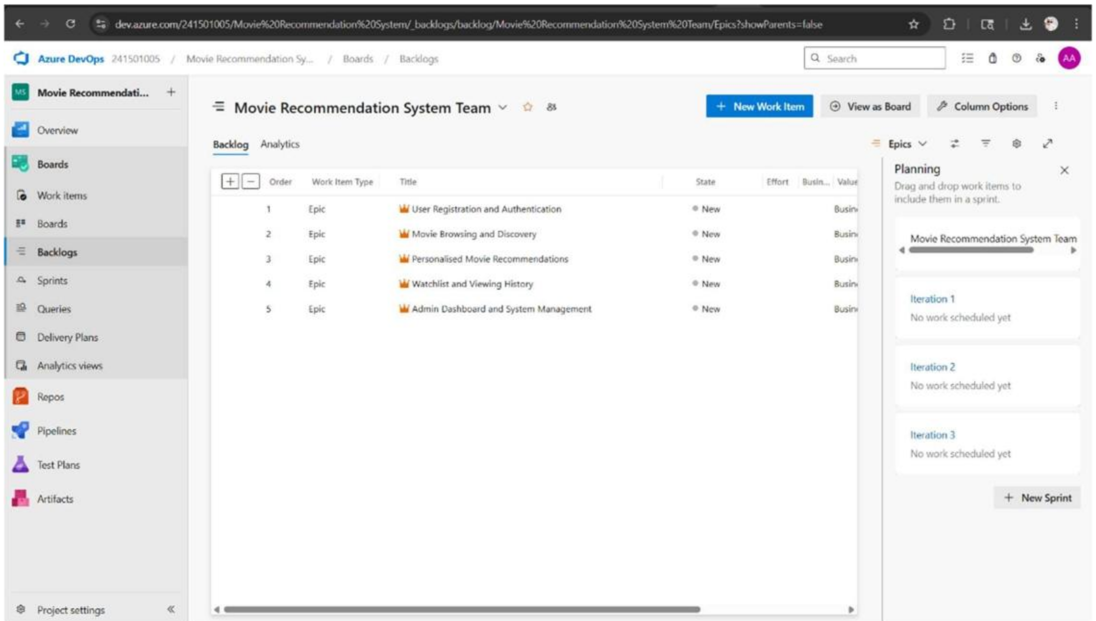
4.Project dashboard

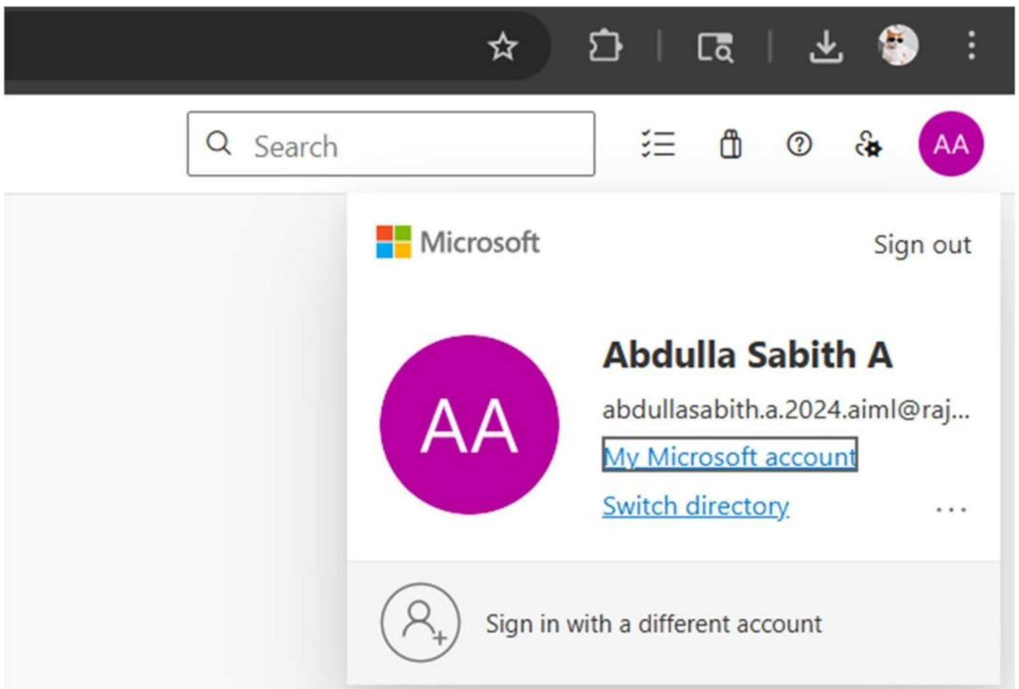


5.To manage user stories:

a. From the **left-hand navigation menu**, click on **Boards**. This will take you to the main **Boards** page, where you can manage work items, backlogs, and sprints.

b. On the **work items** page, you'll see the option to **Add a work item** at the top. Alternatively, you can find a + button or Add New Work Item depending on the view you're in. From the **Add a work item** dropdown, select **User Story**. This will open a form to enter details for the new User Story.





Result:

Successfully created an Azure DevOps project with user story management and agile workflow setup.

Question:
In Azure platform, with a team of 4 people plan a movie recommendation system in a span of 2 sprints which includes complete user stories for all major features, including both functional (e.g., user login, movie browsing, personalised recommendations) and non-functional (e.g., reliability, scalability, performance) requirements.
Portrait the Architecture diagram / Sequence diagram / ER diagram / Class diagram in the Azure platform.
Work on the Test Cases for key functionalities and monitor test results and bug reports in Azure Test Plans. (will be updated after completion of the unit 4 and the unit 5)

MOVIE RECOMMENDATION SYSTEM

EPIC 1: User Registration and Authentication

Feature 1.1: User Registration

User Story 1.1a

As a user, I want to register on the Movie Recommendation System with my credentials using a strong password (minimum 8 characters, 1 numeral and 1 special character), so that I can access my personalised movie recommendations securely.

Task/Bug 1.1a:

- Design registration UI form.

Implement backend validation for strong **password**.

- Integrate password encryption (hashing).
- Create user table in Azure SQL Database.
- Send email verification link.
- Bug: Password validation not triggering for special characters.
- Bug: Duplicate email registration allowed.

User Story 1.1b

As a user, I want to receive an email verification link after registration so that my account is authenticated and protected from unauthorised access.

Task/Bug 1.1b:

- Configure email service (SMTP/Azure Communication Service).
- Generate unique verification token.
- Set token expiry (24 hours).
- Update account status after verification.
- Bug: Verification link expired immediately.
- Bug: Email sent to spam folder.

Feature 1.2: Login / Logout

User Story 1.2a

As a developer, I want to provide login to the Movie Recommendation System within 3 seconds, so that users can quickly access their personalised content.

Task/Bug 1.2a:

- Optimise database queries.
- Enable caching for user session data.
- Implement JWT-based session management.

- Monitor response time using Azure Monitor.
- Bug: Login takes more than 5 seconds during peak load.
- Bug: Server timeout issue.

User Story 1.2b

As a user, I want to log in to the Movie Recommendation System in a single step, so that I can quickly access my recommendations and watchlist.

Task/Bug 1.2b:

- Design simple login UI.
- Enable remember-me option.
- Implement JWT authentication.
- Provide logout functionality.
- Bug: Session not cleared after logout.
- Bug: Invalid credentials error not displayed properly.

EPIC 2: Movie Browsing and Discovery

Feature 2.1: Browse Movies by Genre / Language

User Story 2.1

As a user, I want to browse movies by genre, language, and release year so that I can discover films that match my taste.

Task/Bug 2.1:

- Build genre and language filter APIs.
- Store movie metadata (genre, language, year) in Azure SQL Database.
- Enable search and filter options on the movie listing page.
- Bug: Genre filter not applying correctly on page reload.

User Story 2.2

As a user, I want to filter movies by rating, language, and release year so that I can find movies that suit my preferences.

Task/Bug 2.2:

- Implement multi-select filter functionality.
- Sort movies by TMDB rating score.
- Add language and genre category tags.
- Bug: Filter state resets after page refresh.

Feature 2.2: Movie Details Page

User Story 2.2.1

As a user, I want to view detailed movie information including poster, cast, synopsis and rating so that I can decide whether to watch it.

Task/Bug 2.2.1:

- Fetch movie poster and metadata from TMDB API.
- Display cast, director, synopsis and TMDB rating.
- Bug: Movie poster not loading for some titles.

User Story 2.2.2

As a user, I want to see if a movie is available on popular streaming platforms so that I know where to watch it.

Task/Bug 2.2.2:

- Add streaming availability field in database.

- Display streaming platform badges (e.g. Netflix, Prime).
- Bug: Incorrect streaming platform shown for some movies.

EPIC 3: Personalised Movie Recommendations

Feature 3.1: Recommendation Engine

User Story 3.1

As a user, I want to receive movie recommendations based on my watch history and preferences so that I discover films I will enjoy.

Task/Bug 3.1:

- Build collaborative filtering recommendation model.
- Store user ratings and watch history in database.
- Generate top-10 personalised recommendations per user.
- Bug: Recommendations not updating after new ratings are added.

User Story 3.2

As a user, I want to receive mood-based movie recommendations so that I can find films suited to how I feel right now.

Task/Bug 3.2:

- Build mood selection UI (e.g. Happy, Thriller, Romantic).
- Map mood tags to movie genres in the backend.
- Return filtered recommendations based on selected mood.
- Bug: Mood filter returning irrelevant genres.

Feature 3.2: Similar Movie Suggestions

User Story 3.2.1

As a user, I want to see a list of similar movies on any movie's detail page so that I can find related content easily.

Task/Bug 3.2.1:

- Implement content-based filtering using genre and cast similarity.
- Fetch similar movie data from TMDB API.
- Display similar movies as a horizontal carousel on the detail page.
- Bug: Similar movies list showing unrelated titles.

User Story 3.2.2

As a user, I want to receive weekly email recommendations based on my recent activity so that I stay updated on new films.

Task/Bug 3.2.2:

- Build a scheduled Azure Function to send weekly recommendation emails.
- Update order status to "Paid".
- Bug: Email not triggered for users with no recent activity.

EPIC 4: Watchlist and Viewing History

Feature 4.1: Watchlist Management

User Story 4.1

As a user, I want to add and remove movies from my watchlist so that I can keep track of films I plan to watch.

Task/Bug 4.1:

- Create watchlist table in Azure SQL Database.

- Build add/remove watchlist API endpoints.
- Display watchlist on user profile page.
- Bug: Watchlist not persisting after session logout.

User Story 4.2

As a user, I want to rate and review movies I have watched so that I can share my opinion and improve my recommendations.

Task/Bug 4.2:

- Build star-rating and review submission UI.
- Store ratings and reviews in the database.
- Bug: Rating submission failing silently for some users.

Feature 4.2: Viewing History

User Story 4.2.1

As a user, I want to view my complete movie-watching history so that I can revisit films I have seen before.

Task/Bug 4.2.1:

- Log each movie viewed to the history table in the database.
- Display watch history with dates on user profile.
- Bug: Duplicate entries appearing in watch history.

User Story 4.2.2

As a user, I want to mark movies as “Watched” from the recommendation page so that the system can refine future suggestions.

Task/Bug 4.2.2:

- Build “Mark as Watched” button on movie cards.
- Update user history and re-trigger recommendation recalculation.
- Bug: Watched status not reflected immediately on UI.

EPIC 5: Admin Dashboard and System Management

Feature 5.1: User Feedback and Ratings Management

User Story 5.1

As a user, I want to provide ratings and written feedback on movies so that I can contribute to the community and help others decide.

Task/Bug 5.1:

- Build feedback form with star rating and text comment field.
- Store ratings and reviews in database linked to user and movie.
- Calculate and display average rating per movie.
- Bug: Average rating not updating after new submissions.

User Story 5.2

As an admin, I want to review and moderate user reviews so that inappropriate content is removed and service quality is maintained.

Task/Bug 5.2:

- Build admin review moderation panel in Azure DevOps.
- Flag and filter reviews below a quality threshold.
- Bug: Reviews not displaying in chronological order.

Feature 5.2: Movie Data and Report Management

User Story 5.2.1

As an admin, I want to sync the latest movie data from TMDB API on a scheduled basis so that the platform always shows up-to-date content.

Task/Bug 5.2.1:

- CRUD operations for menu.
- Update price & availability.
- Bug: Changes not reflected immediately.

User Story 5.2.2

As an admin, I want to generate sales reports, so that I can analyze business performance.

Task/Bug 5.2.2:

- Build Azure Function to fetch new movie data daily from TMDB API.
- Export platform usage reports as PDF or Excel.
- Bug: Scheduled sync job failing silently on API timeout.

MOVIE RECOMMENDATION SYSTEM USING MICROSOFT AZURE

(Agile – 2 Sprints | Team of 4 | Architecture + Test Plans)

Agile Planning (2 Sprints)

Team Structure (4 Members)

Role	Responsibility
Scrum Master	Sprint planning, daily standups, removing blockers
Backend Developer	API, database, recommendation engine logic
Frontend Developer	UI (Web/App), user experience, movie browsing pages
QA/Test Engineer	Test cases, Azure Test Plans, bug tracking

Sprint Planning Overview

Sprint 1 (Week 1–2)

Goal: Core user and movie browsing functionalities

Features:

- User Registration & Login
- Movie listing with genre and language filters
- Movie detail page with TMDB metadata
- Watchlist management (add/remove)

- User ratings and review submission
- Basic recommendation engine (genre-based)
- Azure SQL Database setup (Users, Movies, Ratings, Watchlist)
- Deployment on Azure App Service

Sprint 2 (Week 3–4)

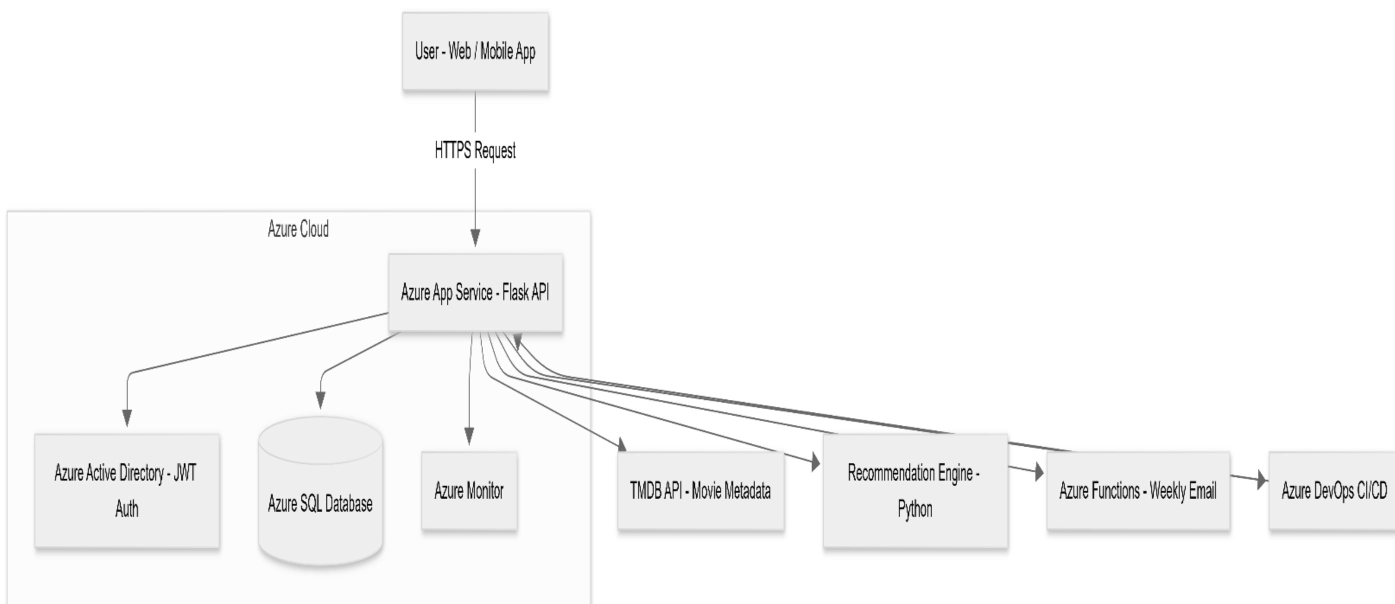
Goal: Advanced recommendations & Quality

Features:

- Personalised recommendation engine (collaborative filtering)
- Admin dashboard for data and review management
- Viewing history and watched status tracking
- Weekly email recommendations via Azure Functions
- Performance optimisation and caching
- Security enhancements (rate limiting, input validation)
- Load testing with Azure Load Testing
- Bug fixing
- CI/CD pipeline setup

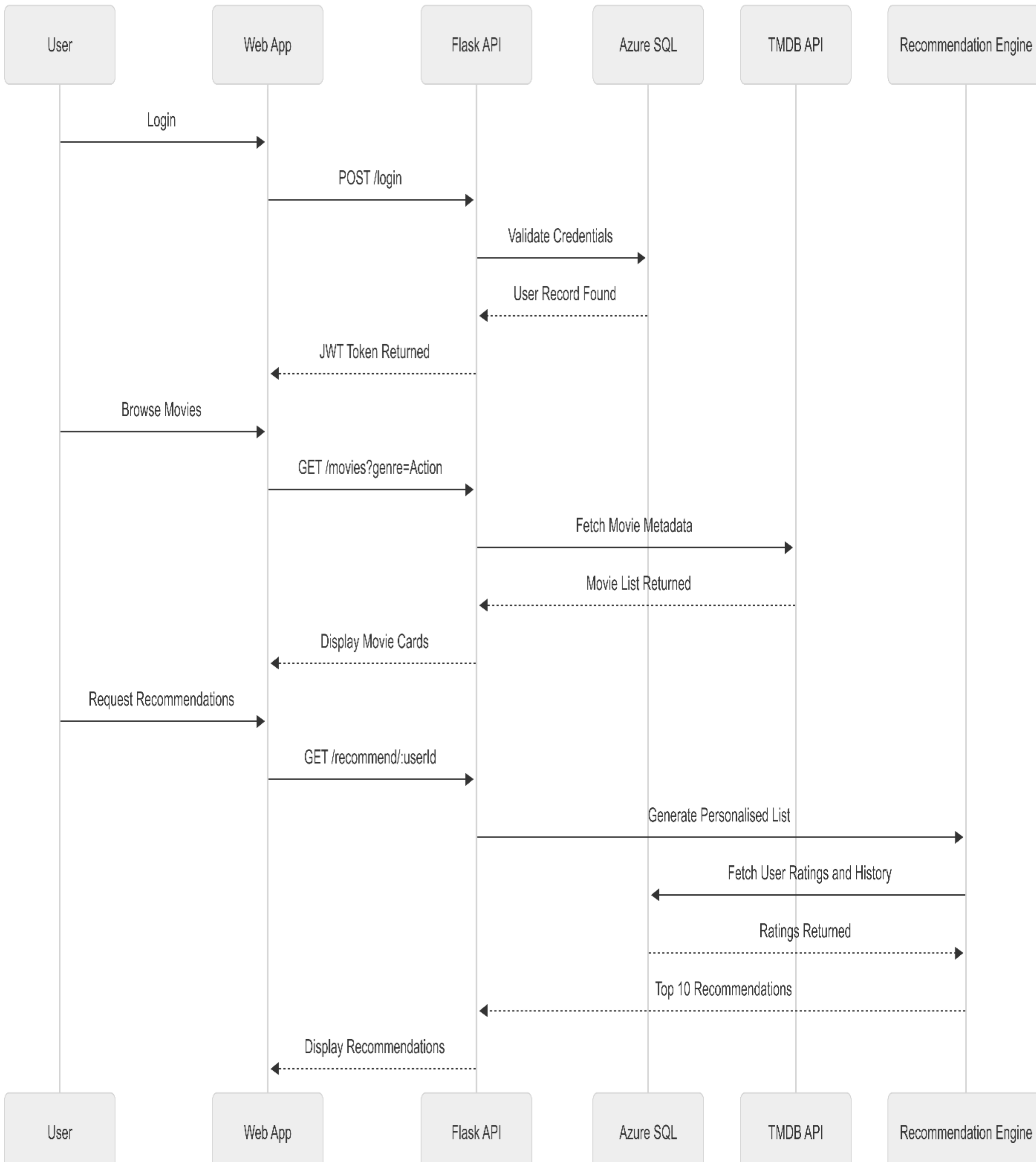
Architecture Diagram:

```
flowchart TD

User[User - Web / Mobile App] -->|HTTPS Request| AppService[Azure App Service - Flask API]

AppService --> Auth[Azure Active Directory - JWT Auth]
AppService --> DB[(Azure SQL Database)]
AppService --> TMDB[TMDB API - Movie Metadata]
AppService --> RecEngine[Recommendation Engine - Python]
AppService --> AzFunc[Azure Functions - Weekly Email]

AppService --> Monitor[Azure Monitor]
AppService --> DevOps[Azure DevOps CI/CD]
DevOps --> AppService

subgraph Azure Cloud
   AppService
   DB
   Auth
   Monitor
end
```



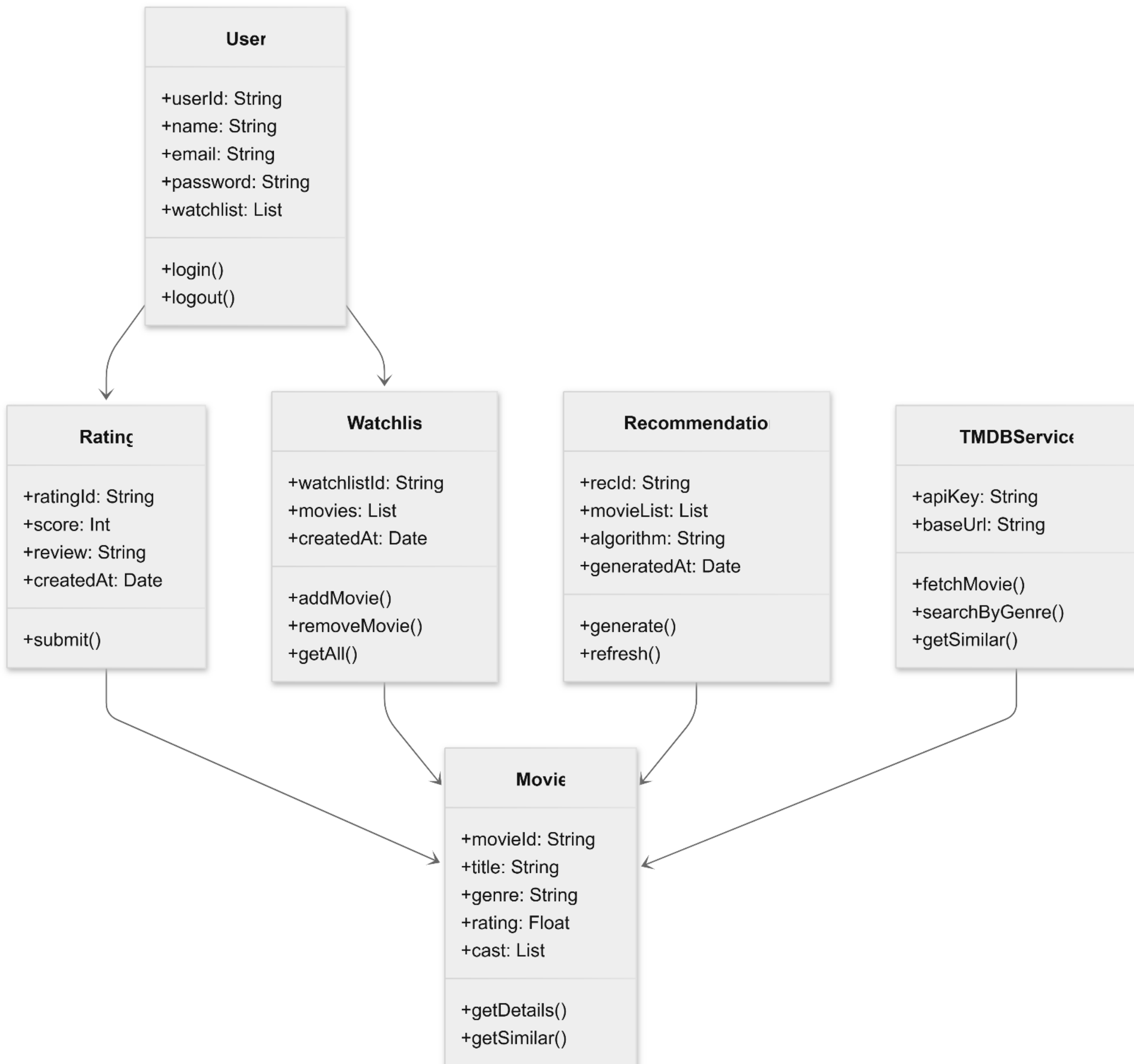
Sequence Diagram

```
sequenceDiagram

participant User
participant UI as Web App
participant API as Flask API
participant DB as Azure SQL
participant TMDB as TMDB API
participant Rec as Recommendation Engine

User->>UI: Login
UI->>API: Authenticate Request
API->>DB: Validate Credentials
DB-->>API: User Record Found
API-->>UI: JWT Token Returned

User->>UI: Browse Movies / Apply Filters
UI->>API: GET /movies?genre=Action
API->>TMDB: Fetch Movie Metadata

TMDB-->>API: Movie List Returned
API-->>UI: Display Movie Cards
User->>UI: Request Recommendations
UI->>API: GET /recommend/:userId
API->>Rec: Generate Personalised List
Rec->>DB: Fetch User Ratings and History

DB-->>Rec: Ratings Returned
Rec-->>API: Top 10 Movie Recommendations
```


User
Web App
Flask API
Azure SQL
TMDB API
Recommendation Engine
Login
POST /login
Validate Credentials
User Record Found
JWT Token Returned
Browse Movies
GET /movies?genre=Action
Fetch Movie Metadata
Movie List Returned
Display Movie Cards
Request Recommendations
GET /recommend/:userId
Generate Personalised List
Fetch User Ratings and History
Ratings Returned
Top 10 Recommendations
Display Recommendations
User
Web App
Flask API
Azure SQL
TMDB API
Recommendation Engine

Class Diagram

```
API
classDiagram

class User {
  +userId: String
  +name: String
  +email: String
  +password: String
  +watchlist: List
  +login()
  +logout()
}

class Movie {
  +movieId: String
  +title: String
  +genre: String
  +rating: Float
  +cast: List
  +getDetails()
  +getSimilar()
}

class Rating {
  +ratingId: String
  +score: Int
  +review: String
  +createdAt: Date
  +submit()
}
class Watchlist {
  +watchlistId: String
  +movies: List
  +createdAt: Date
  +addMovie()
  +removeMovie()
  +getAll()
}

class Recommendation {
  +recId: String
  +movieList: List
  +algorithm: String
  +generatedAt: Date
  +generate()
  +refresh()
}

class TMDBService {
  +apiKey: String
  +baseUrl: String
  +fetchMovie()
  +searchByGenre()
  +getSimilar()
}

User --> Rating
User --> Watchlist
Rating --> Movie
Watchlist --> Movie
Recommendation --> Movie
TMDBService --> Movie
```

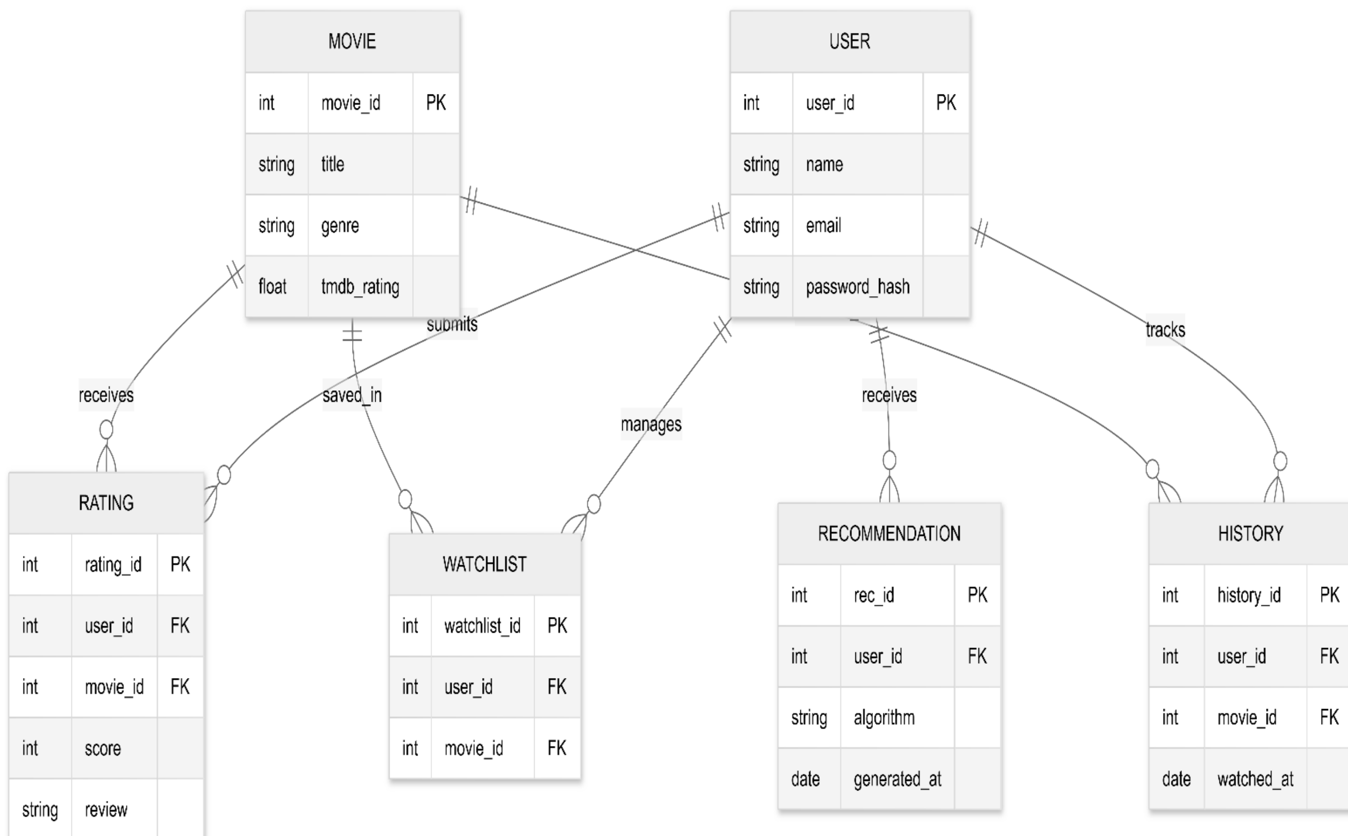

User
+userId: String
+name: String
+email: String
+password: String
+watchlist: List
+login()
+logout()
Ratinç
+ratingId: String
+score: Int
+review: String
+createdAt: Date
+submit()
Watchlis
+watchlistId: String
+movies: List
+createdAt: Date
+addMovie()
+removeMovie()
+getAll()
Recommendatio
+recId: String
+movieList: List
+algorithm: String
+generatedAt: Date
+generate()
+refresh()
TMDBServic
+apiKey: String
+baseUrl: String
+fetchMovie()
+searchByGenre()
+getSimilar()
Movie
+movieId: String
+title: String
+genre: String
+rating: Float
+cast: List
+getDetails()
+getSimilar()

ER Diagram

A. Crow's Foot style:

- Entities are shown as boxes
- Attributes are listed inside
- Relationships use symbols like ||--o{

```
erDiagram

USER {
   int user_id PK
   string name
   string email
   string password_hash
}

MOVIE {
   int movie_id PK
   string title
   string genre
   float tmdb_rating
}

RATING {
   int rating_id PK
   int user_id FK
   int movie_id FK
   int score
   string review
}

WATCHLIST {
   int watchlist_id PK
   int user_id FK
   int movie_id FK
}

HISTORY {
   int history_id PK
   int user_id FK
   int movie_id FK
   date watched_at
}

RECOMMENDATION {
   int rec_id PK
   int user_id FK
   string algorithm
   date generated_at
}

USER ||--o{ RATING : submits
USER ||--o{ WATCHLIST : manages
USER ||--o{ HISTORY : tracks
MOVIE ||--o{ RATING : receives
MOVIE ||--o{ WATCHLIST : saved_in
MOVIE ||--o{ HISTORY : watched_in
USER ||--o{ RECOMMENDATION : receives
```


MOVIE
int movie_id PK
string title
string genre
float tmdb_rating
USER
int user_id PK
string name
string email
string password_hash
submits
receives
saved_in
manages
receives
tracks
RATING
int rating_id PK
int user_id FK
int movie_id FK
int score
string review
WATCHLIST
int watchlist_id PK
int user_id FK
int movie_id FK
RECOMMENDATION
int rec_id PK
int user_id FK
string algorithm
date generated_at
HISTORY
int history_id PK
int user_id FK
int movie_id FK
date watched_at

B. DBMS format -**Chen notation**,

- 🔵 Entity → Rectangle
- 🔶 Relationship → Diamond
- ⚪ Attributes → Ovals (circles)
- 🔑 Primary Key → Underlined
- Lines connect entity → relationship → entity
-

flowchart LR

%% Entities
User[USER]
Movie[MOVIE]
Rating[RATING]
Watchlist[WATCHLIST]
History[HISTORY]
Recommendation[RECOMMENDATION]

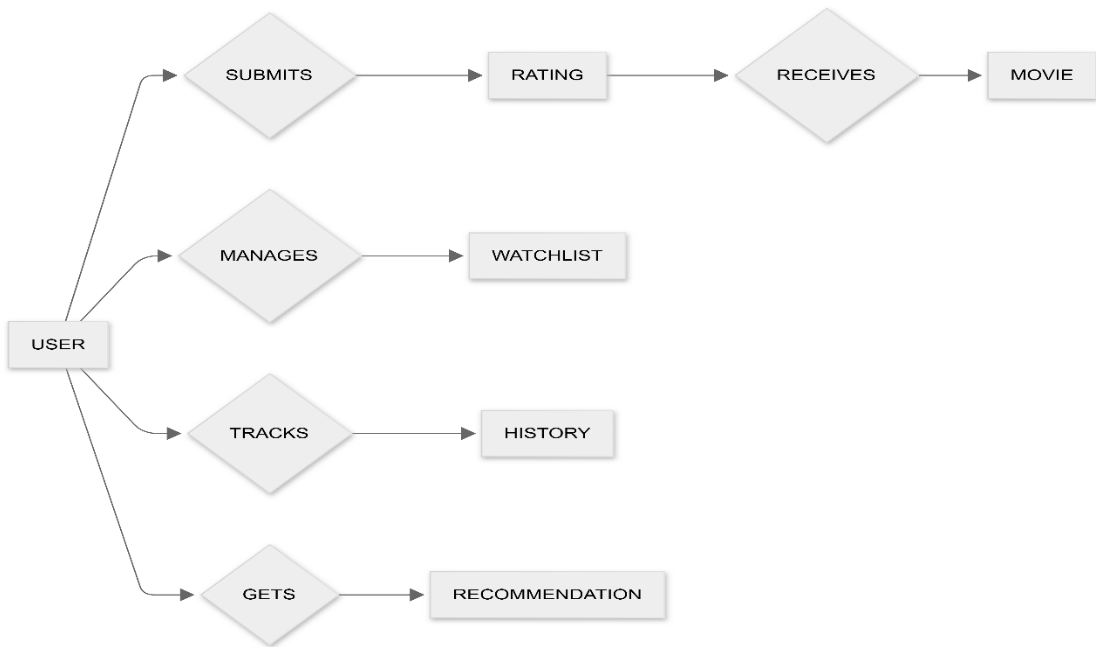
%% Relationships (Diamonds)
Submits{SUBMITS}
Receives{RECEIVES}
Manages{MANAGES}
Tracks{TRACKS}
Gets{GETS}

%% Connections - mirroring the chained structure
User --> Submits
Submits --> Rating
Rating --> Receives
Receives --> Movie

User --> Manages
Manages --> Watchlist

User --> Tracks
Tracks --> History

User --> Gets
Gets --> Recommendation



Use case diagram:

flowchart LR

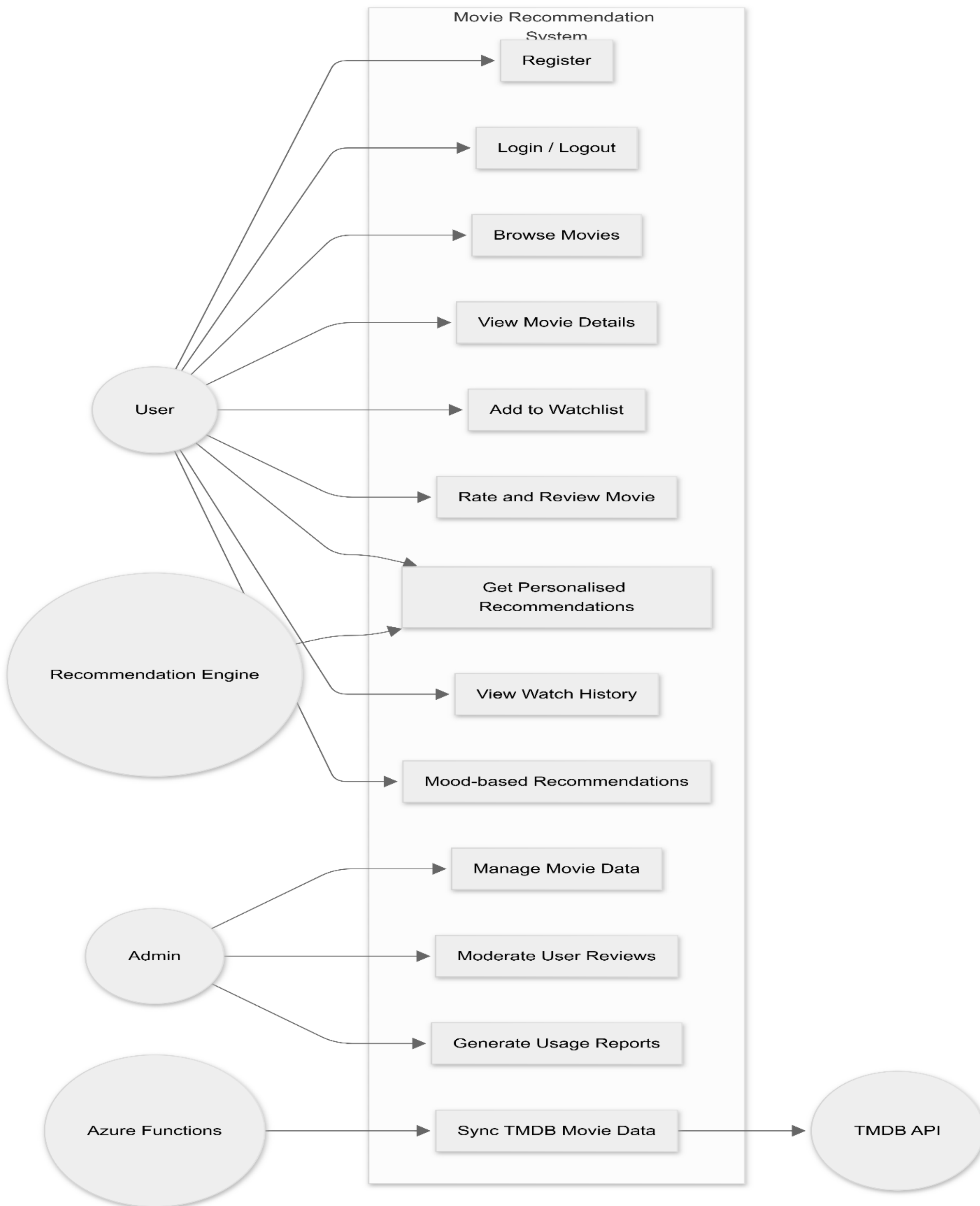
%% Actors
User((User))
Admin((Admin))
RecEngine((Recommendation Engine))
TMDBAPI((TMDB API))
AzureFunc((Azure Functions))

%% System Boundary
subgraph Movie Recommendation System
Register[Register]
Login[Login / Logout]
Browse[Browse Movies]
ViewMovie[View Movie Details]
AddWatchlist[Add to Watchlist]
RateMovie[Rate and Review Movie]
GetRecs[Get Personalised Recommendations]
ViewHistory[View Watch History]
MoodRec[Mood-based Recommendations]
ManageMovies[Manage Movie Data]
ModerateReviews[Moderate User Reviews]
ViewReports[Generate Usage Reports]
SyncTMDB[Sync TMDB Movie Data]
end

%% User actions
User --> Register
User --> Login
User --> Browse
User --> ViewMovie
User --> AddWatchlist
User --> RateMovie
User --> GetRecs
User --> ViewHistory
User --> MoodRec

%% Admin actions
Admin --> ManageMovies
Admin --> ModerateReviews
Admin --> ViewReports

%% System actions
RecEngine --> GetRecs
AzureFunc --> SyncTMDB
SyncTMDB --> TMDBAPI


Movie Recommendation System
Register
Login / Logout
Browse Movies
View Movie Details
User
Add to Watchlist
Rate and Review Movie
Get Personalised Recommendations
Recommendation Engine
View Watch History
Mood-based Recommendations
Manage Movie Data
Admin
Moderate User Reviews
Generate Usage Reports
Azure Functions
Sync TMDB Movie Data
TMDB API

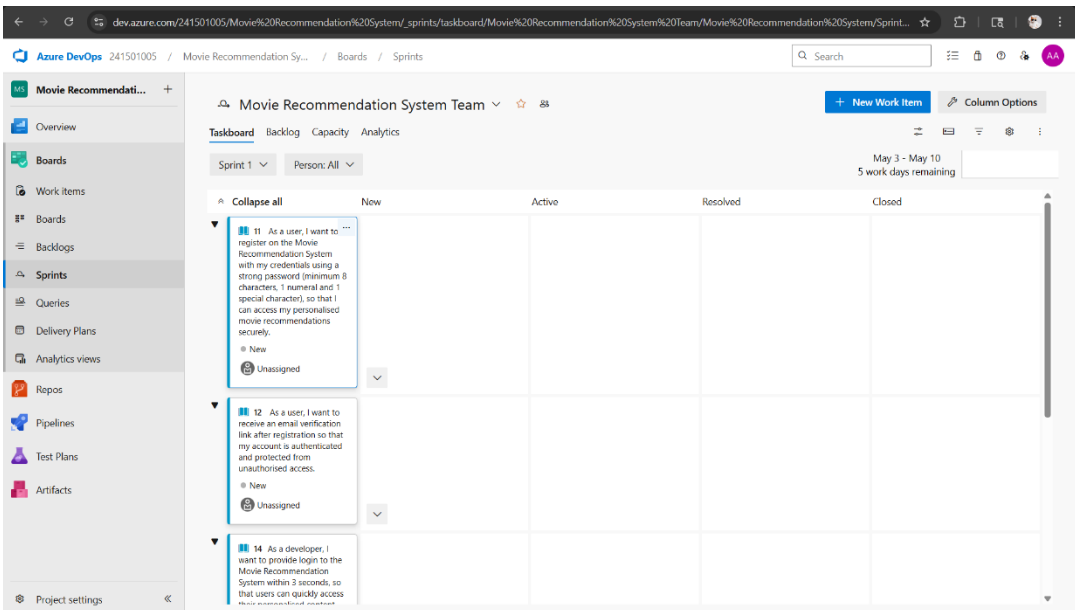
EXP NO: 4	**SPRINT PLANNING**

Aim:

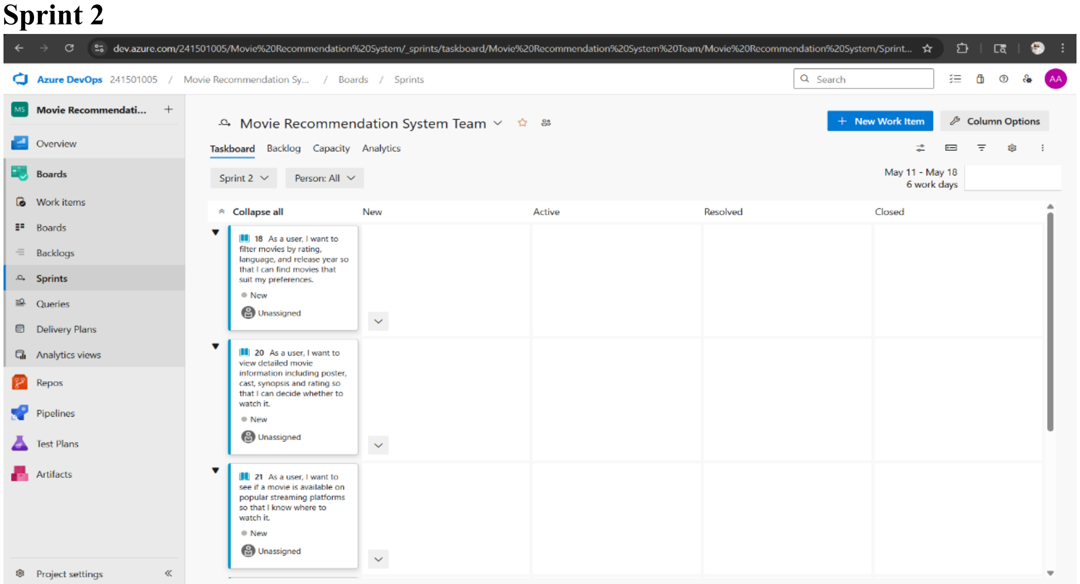
To assign user story to specific sprint for the Movie Recommendation System Project.

Sprint Planning

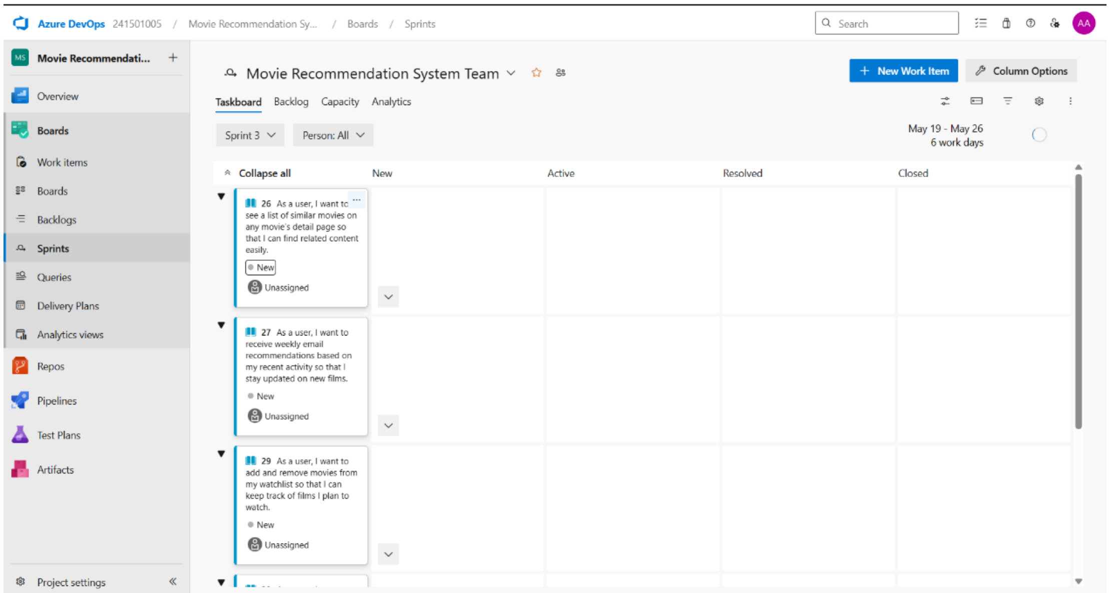
Sprint 1



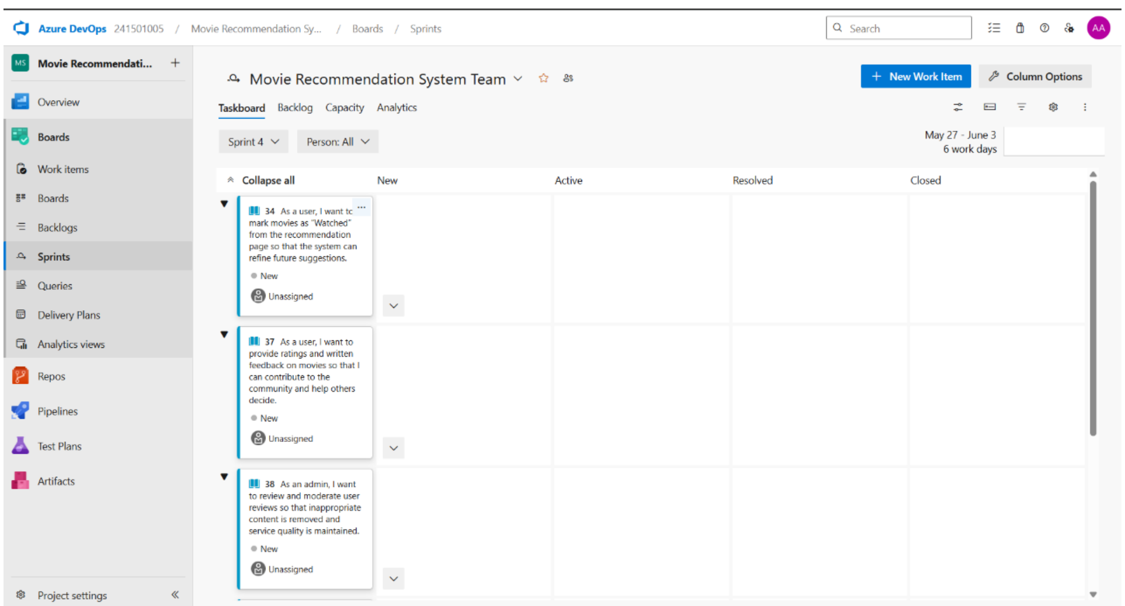
Sprint 2



Sprint 3



Sprint 4



Result:

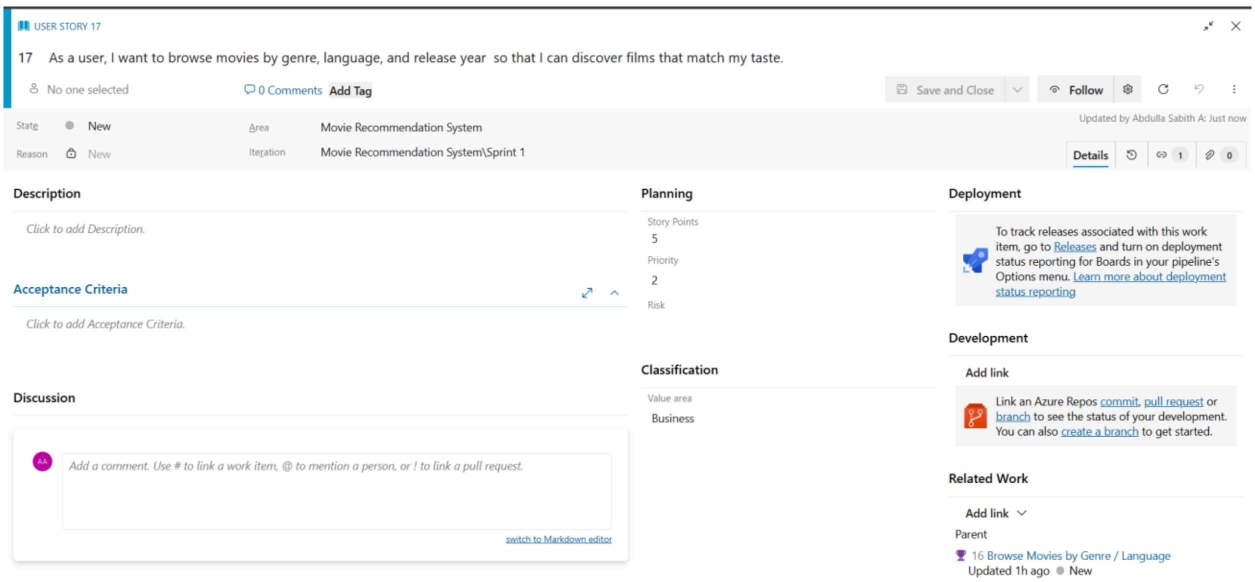
The Sprints are created for the Movie Recommendation System Project.

EXP NO: 5	POKER ESTIMATION

Aim:

Create Poker Estimation for the user stories – Movie Recommendation System Project.

Poker Estimation



Result:

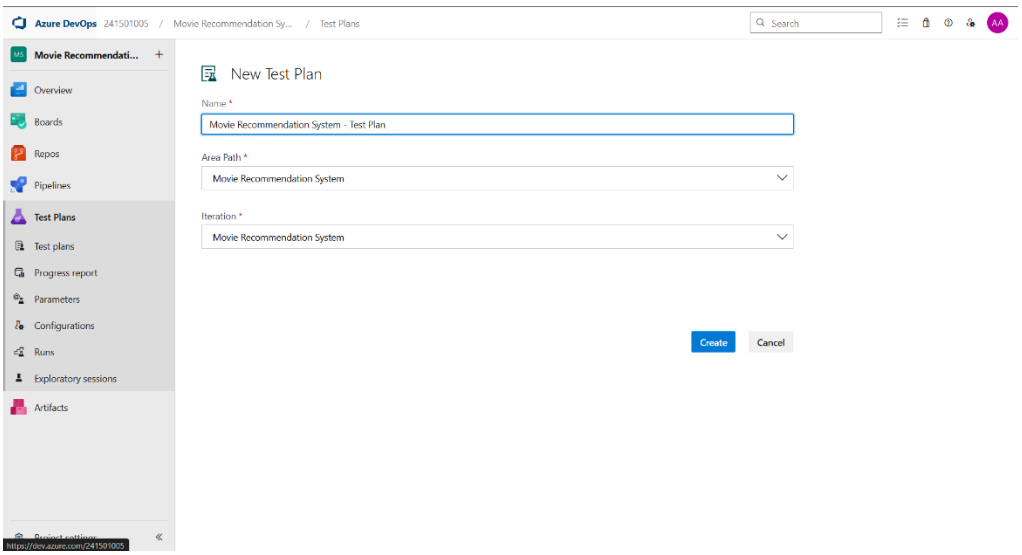
The Estimation/Story Points is created for the project using Poker Estimation.

EXP NO: 8	TESTING – TEST PLANS AND TEST CASES

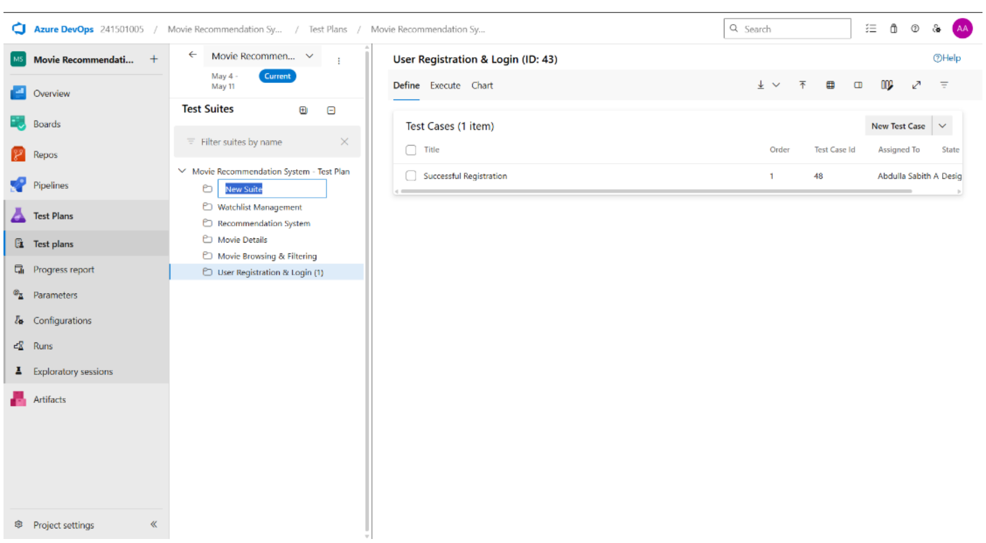
Aim:

Test Plans and Test Case and write two test cases for at least five user stories showcasing the happy path and error scenarios in azure DevOps platform.

1.New test plan



2.Test suite



3.Test case

Give two test cases for at least five user stories showcasing the happy path and error scenarios in azure DevOps platform.

Movie Recommendation System – Test Plans

USER STORIES

- As a user, I want to sign up and log in securely so that I can access my movie recommendations.
- As a user, I want to log in to the Movie Recommendation System in a single step.
- As a user, I want to browse movies by genre, language, and release year.
- As a user, I want to receive movie recommendations based on my watch history and preferences.
- As a user, I want to add and remove movies from my watchlist.

Test Suites

Test Suit: TS01 - User Registration (ID: 86)

1. **TC01 – Successful Registration**
   o Action:
   ▪ Go to the Registration page.
   ▪ Enter valid username, email, and password.
   ▪ Click "Register".

   o Expected Results:
   ▪ Registration page is displayed.
   ▪ Fields accept values without error.
   ▪ Account is created successfully and email verification is sent.

   o Type: Happy Path

2. **TC02 – Weak Password**
   o Action:
   ▪ Go to the Registration page.
   ▪ Enter password without special character.
   ▪ Click "Register".

   o Expected Results:
   ▪ Fields accept values without error.
   ▪ Error message "Password does not meet criteria" is displayed.

   o Type: Error Path

Test Suit: TS02 - Login (ID: 87)

1. **TC03 – Successful Login**
   o Action:
   ▪ Go to the Login page.
   ▪ Enter valid email and password.
   ▪ Click "Login".

   o Expected Results:
   ▪ Login form is displayed.
   ▪ Fields accept data without error.
   ▪ User is logged in and redirected to the dashboard.

   o Type: Happy Path

2. **TC04 – Invalid Credentials**
   o Action:
   ▪ Go to the Login page.
   ▪ Enter wrong password.
   ▪ Click "Login".

   o Expected Results:
   ▪ Fields accept data.
   ▪ Error message "Invalid credentials" is displayed.

   o Type: Error Path

Test Suit: TS03 - Movie Browsing (ID: 88)

1. **TC05 – Browse Movies**
   o Action:
   ▪ Log in successfully.
   ▪ Navigate to Movies section.

   o Expected Results:
   ▪ Movies are displayed clearly.

   o Type: Happy Path

2. **TC06 – Filter Failure**
   o Action:
   ▪ Apply invalid filter.
   ▪ Refresh results.

o Expected Results:
▪ No movies are displayed.
▪ Message "No movies found" is shown.

o Type: Error Path

Test Suit: TS04 - Recommendation System (ID: 89)

1. **TC07 – Personalized Recommendations**
   o Action:
   ▪ Login with valid credentials.
   ▪ Open Recommendations section.

   o Expected Results:
   ▪ Movies are recommended based on user history.

   o Type: Happy Path

2. **TC08 – No History Case**
   o Action:
   ▪ Login as new user.
   ▪ Open Recommendations section.

   o Expected Results:
   ▪ Default recommendations are displayed.

   o Type: Error Path

Test Suit: TS05 - Watchlist Management (ID: 90)

1. **TC09 – Add to Watchlist**
   o Action:
   ▪ Login successfully.
   ▪ Select a movie.
   ▪ Click "Add to Watchlist".

   o Expected Results:
   ▪ Movie is added successfully to watchlist.

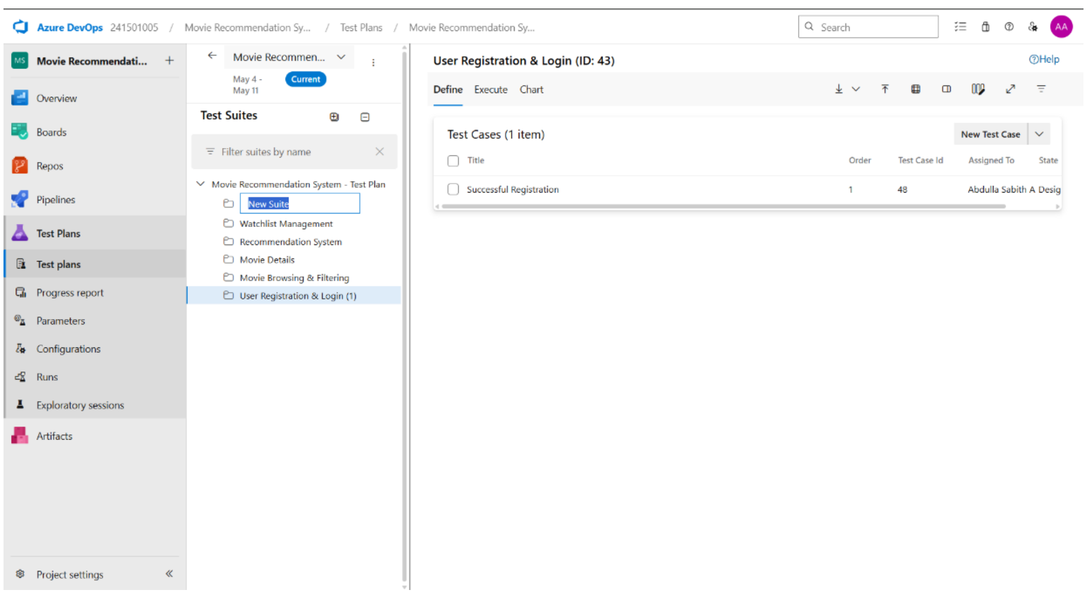
   o Type: Happy Path

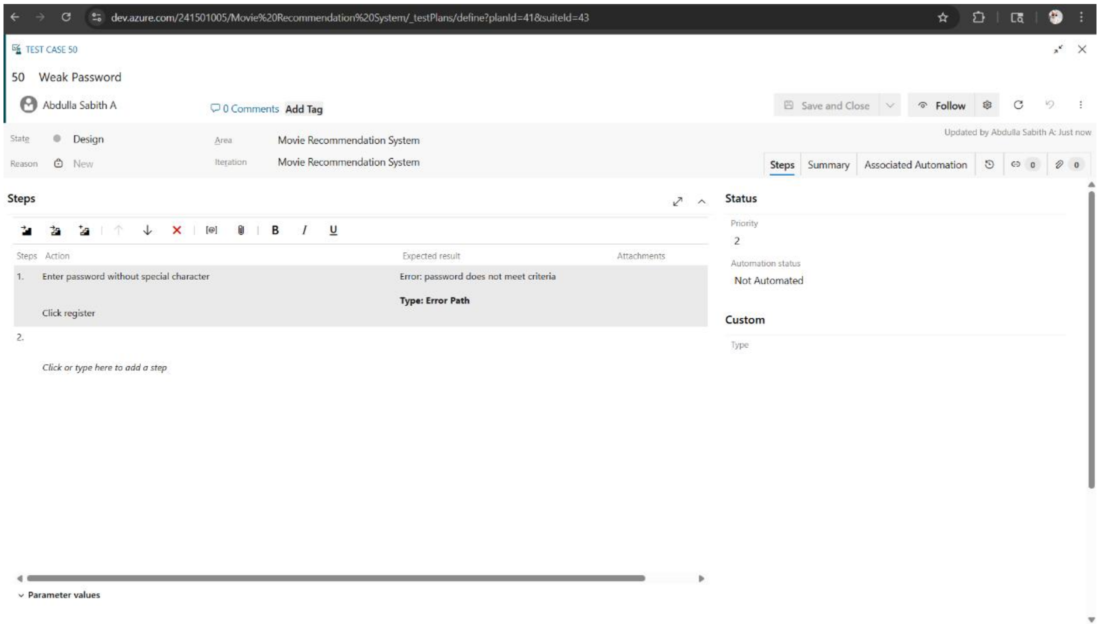
2. **TC10 – Duplicate Add**
   o Action:
   ▪ Select a movie already added.
   ▪ Click "Add to Watchlist" again.

   o Expected Results:
   ▪ Duplicate addition is prevented.
   ▪ Message indicating duplicate is shown.

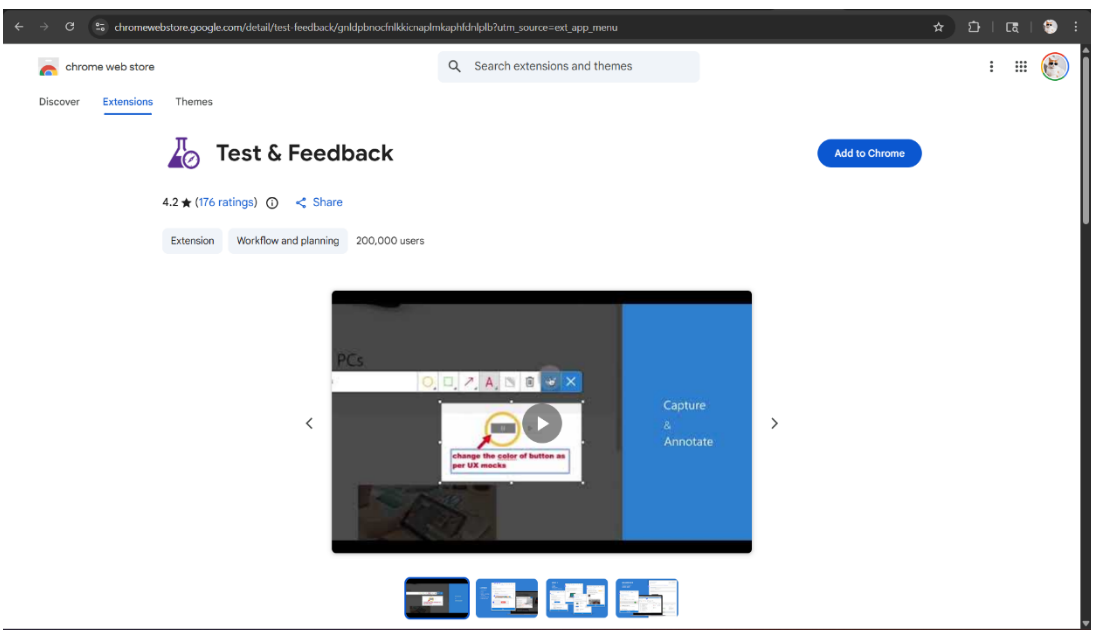
   o Type: Error Path

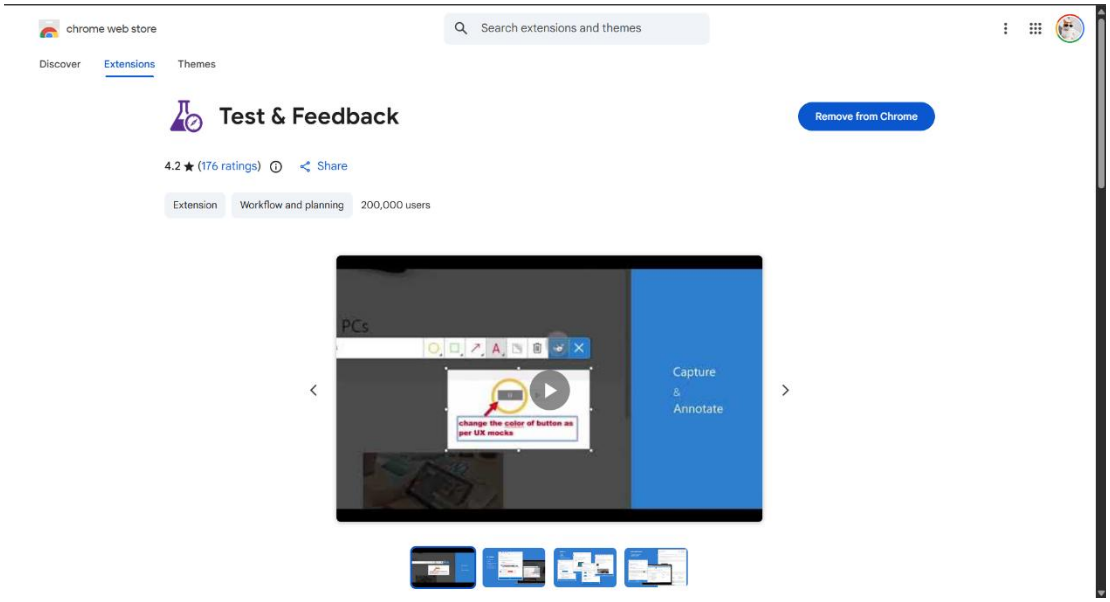
Test Cases



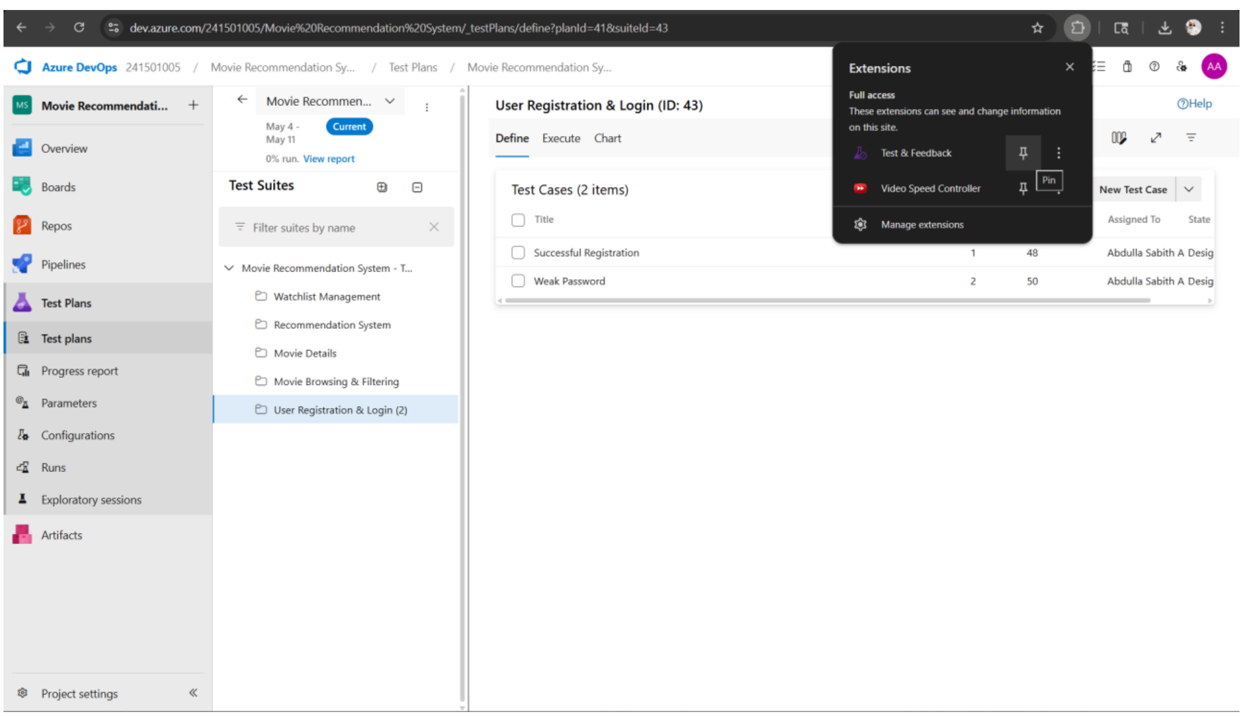


4.Installation of test

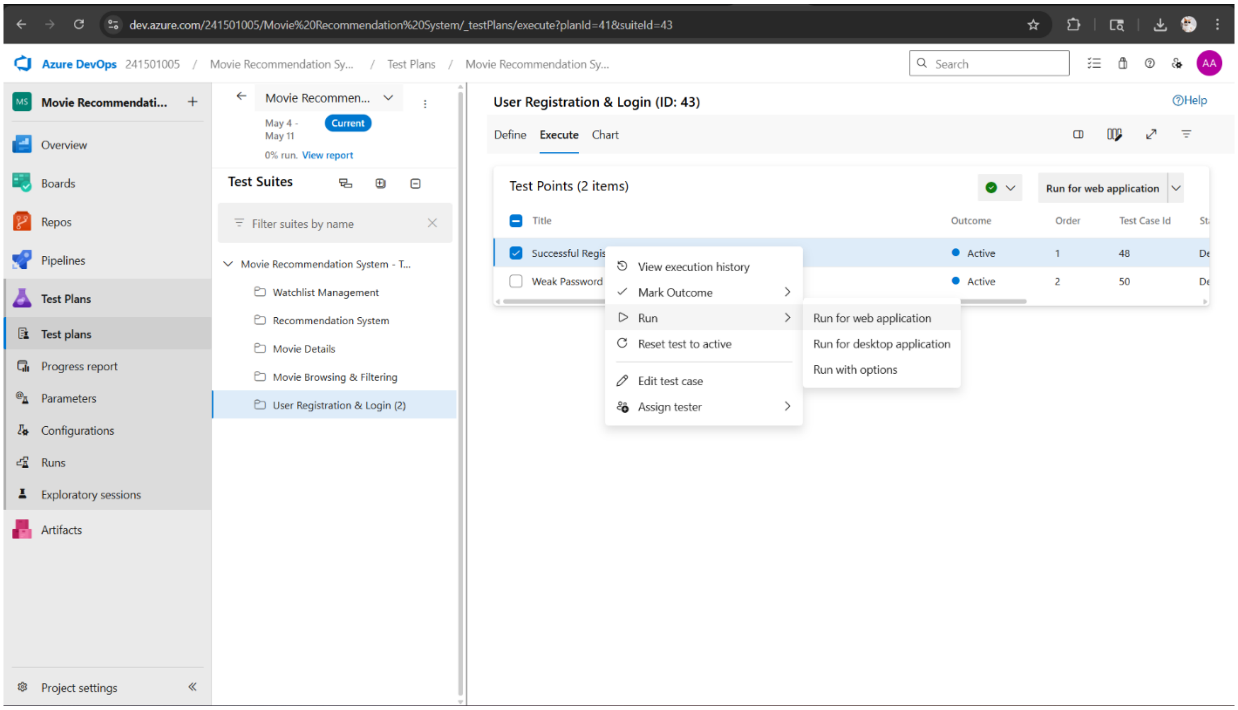


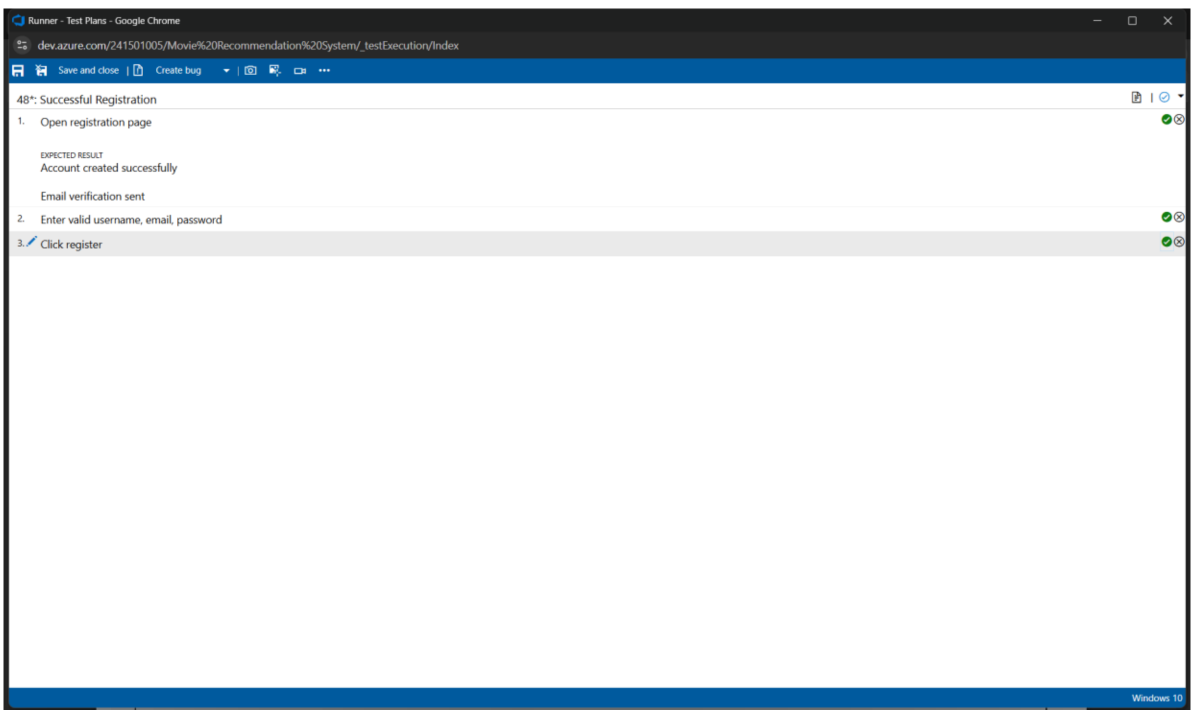


Test and feedback
Showing it as an extension

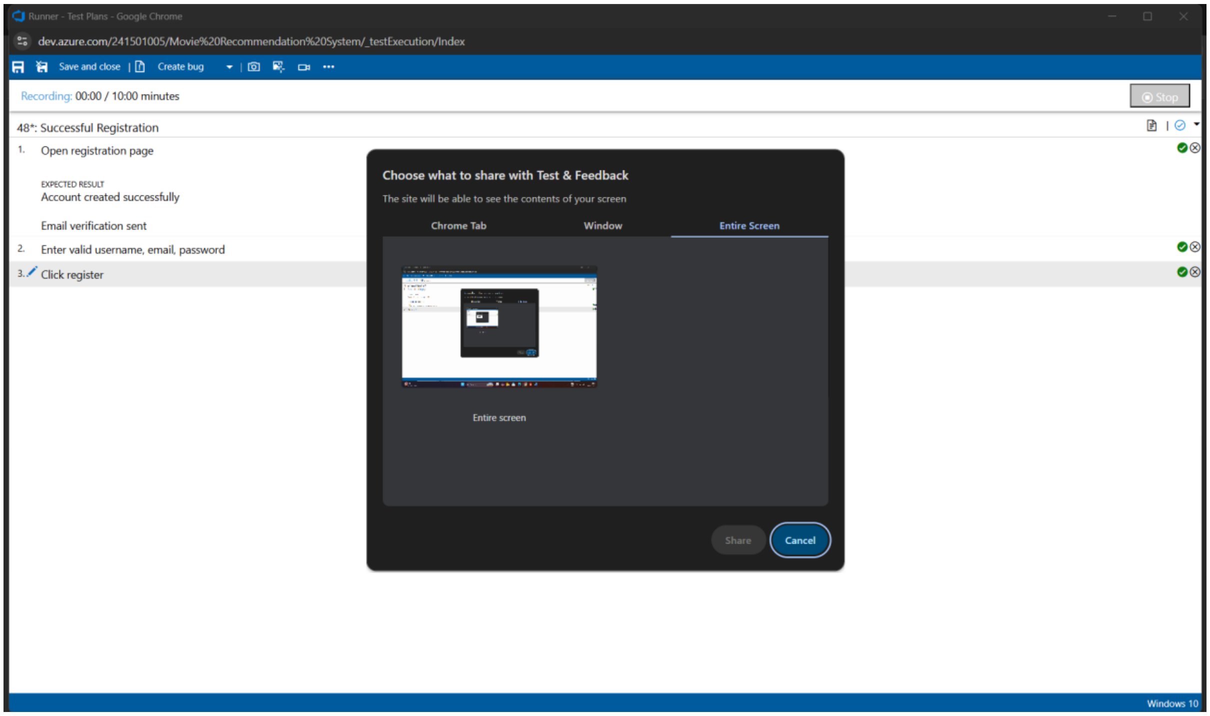


5.Running the test cases

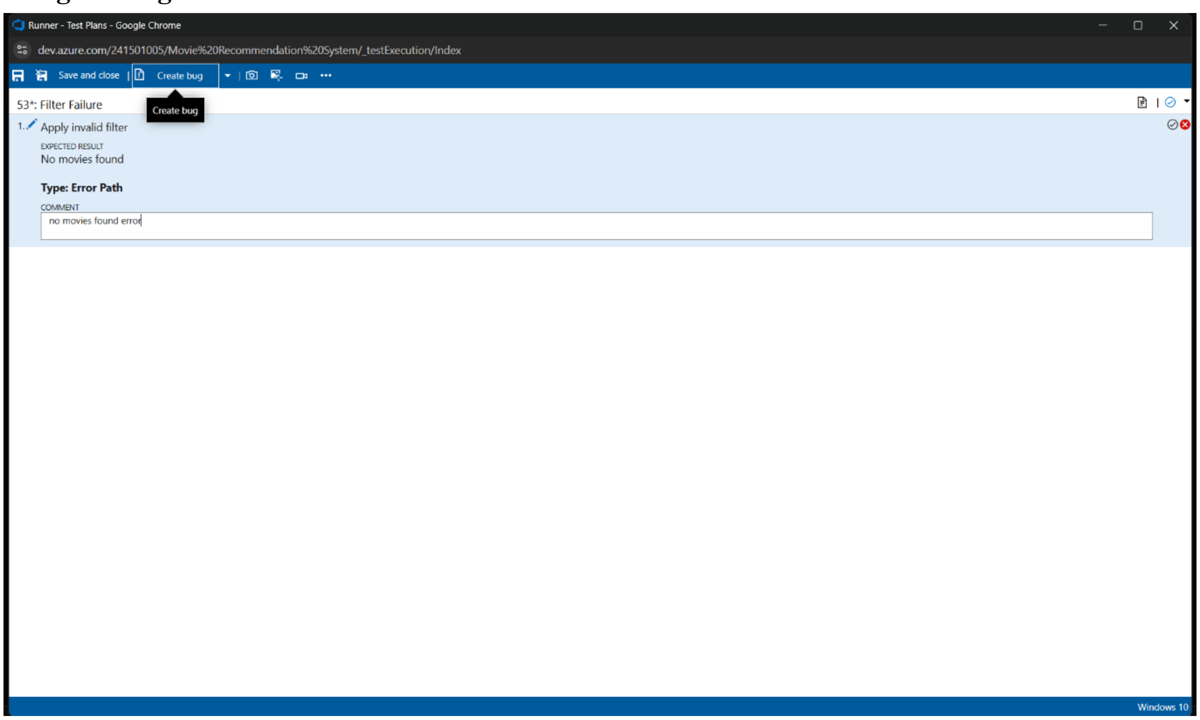


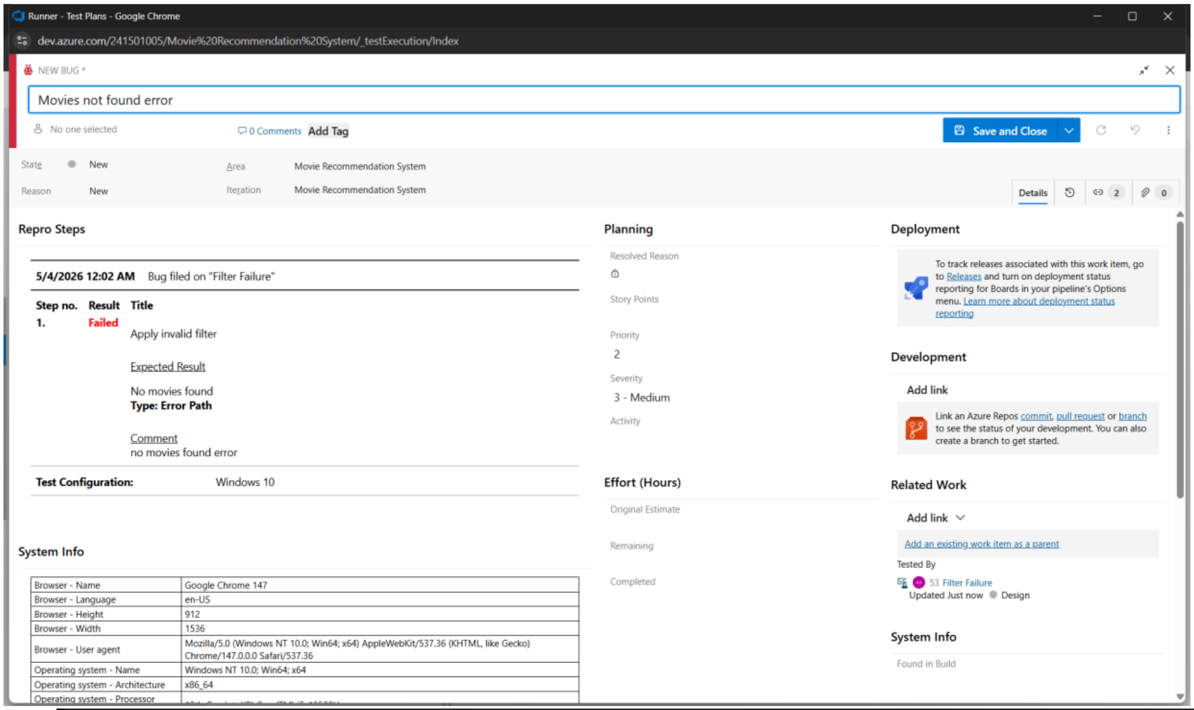


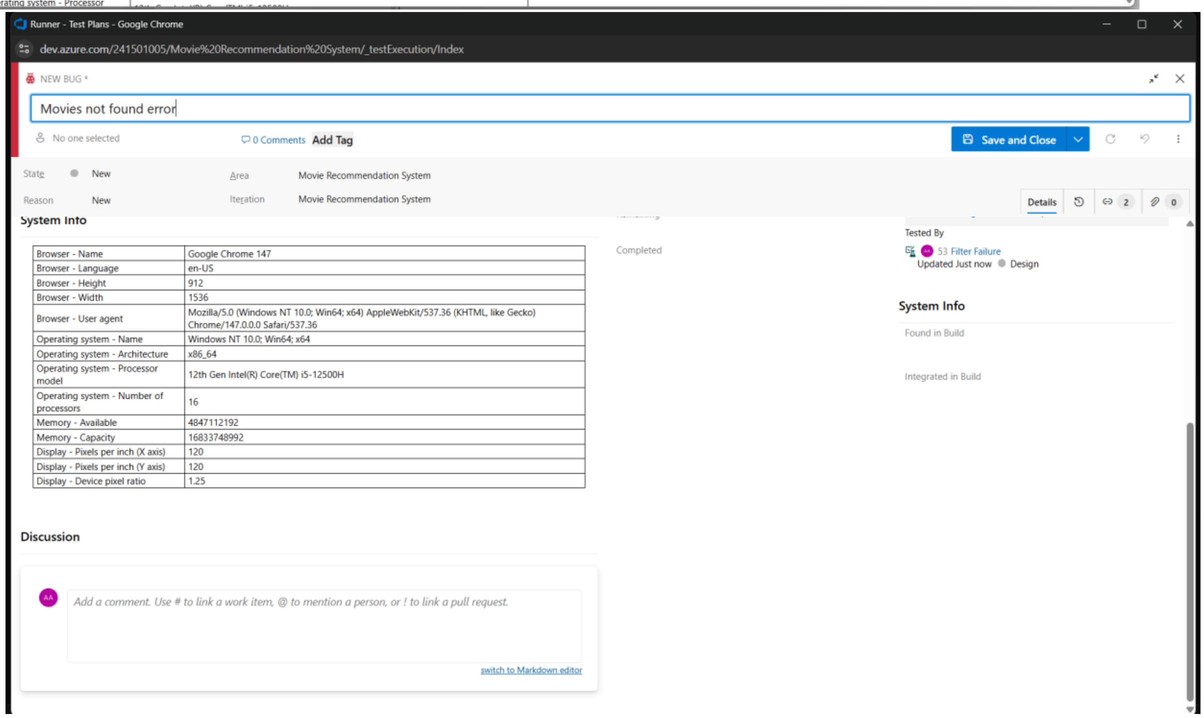
6.Recording the test case



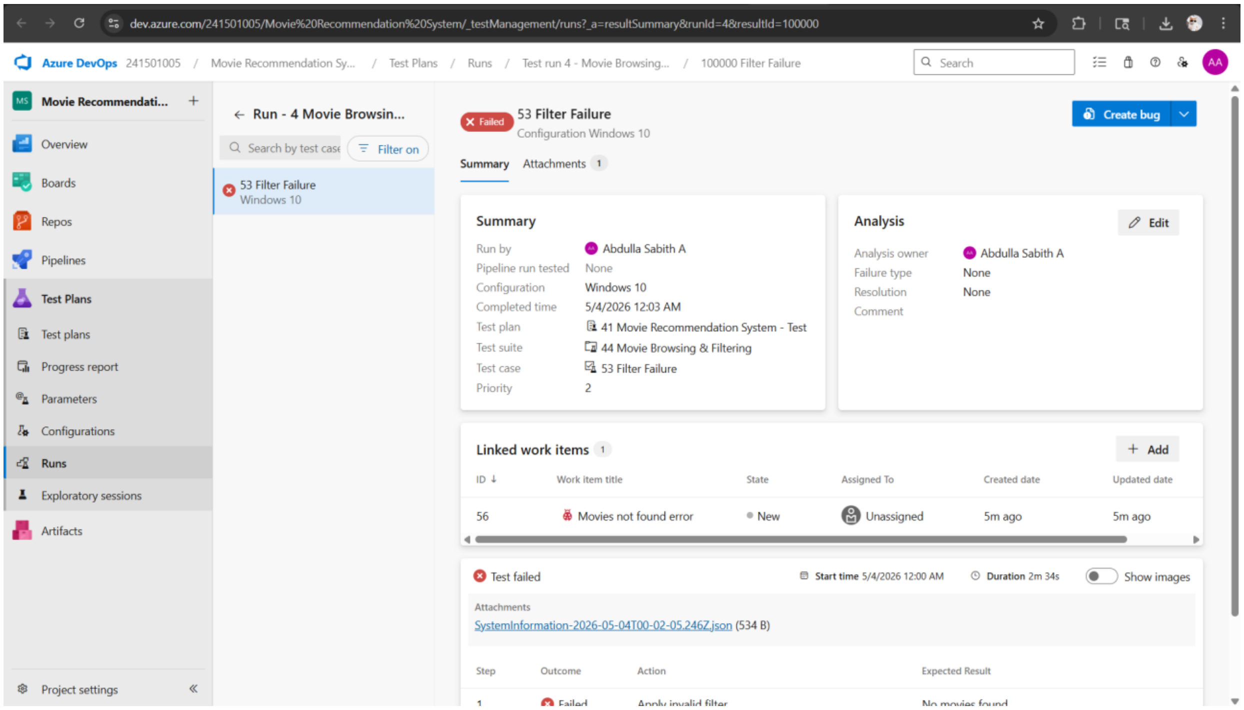
7.Creating the bug



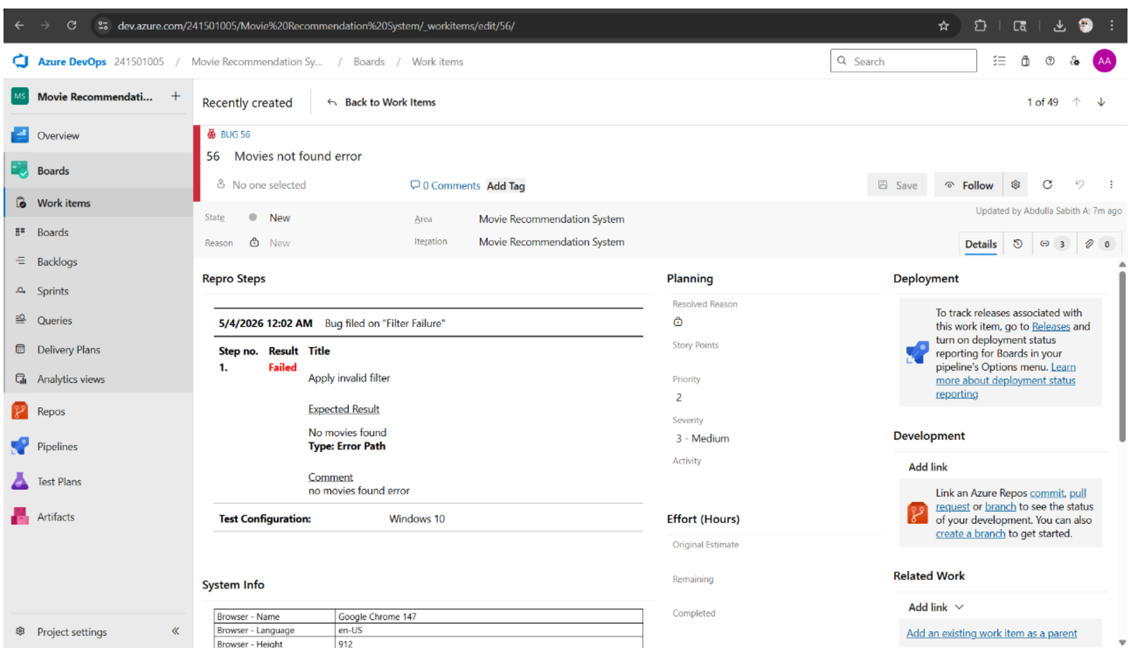




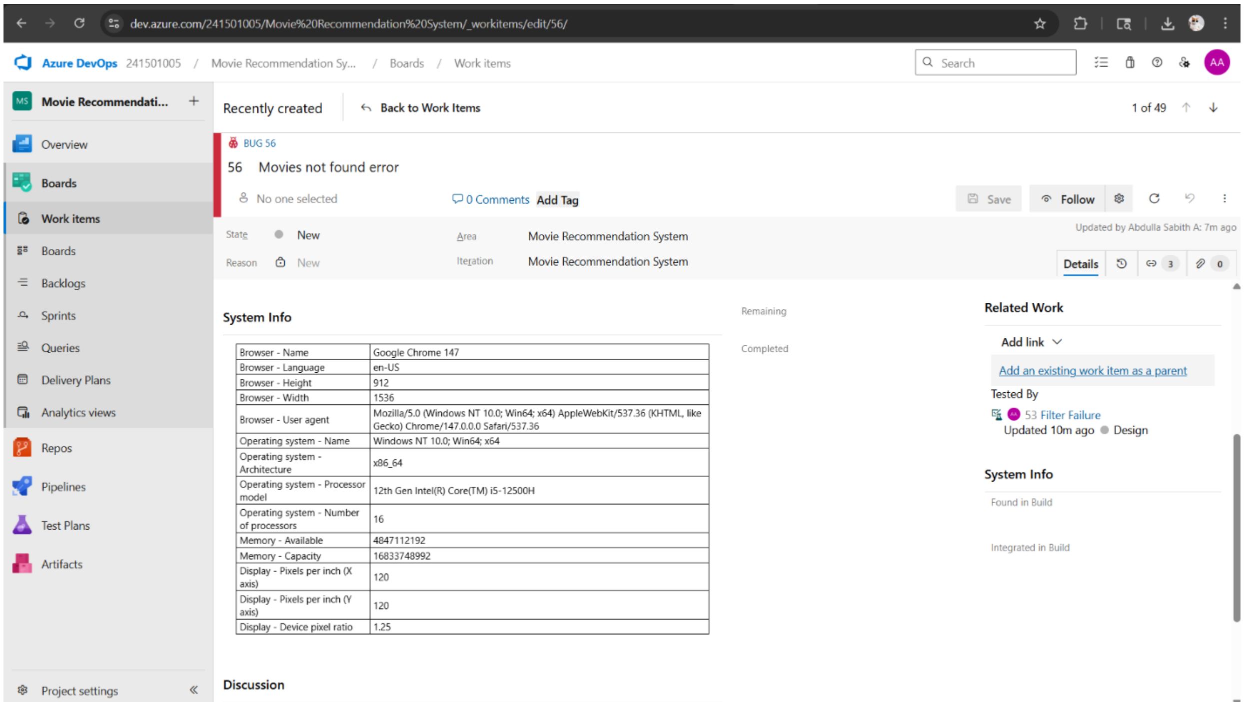
8.Test case results



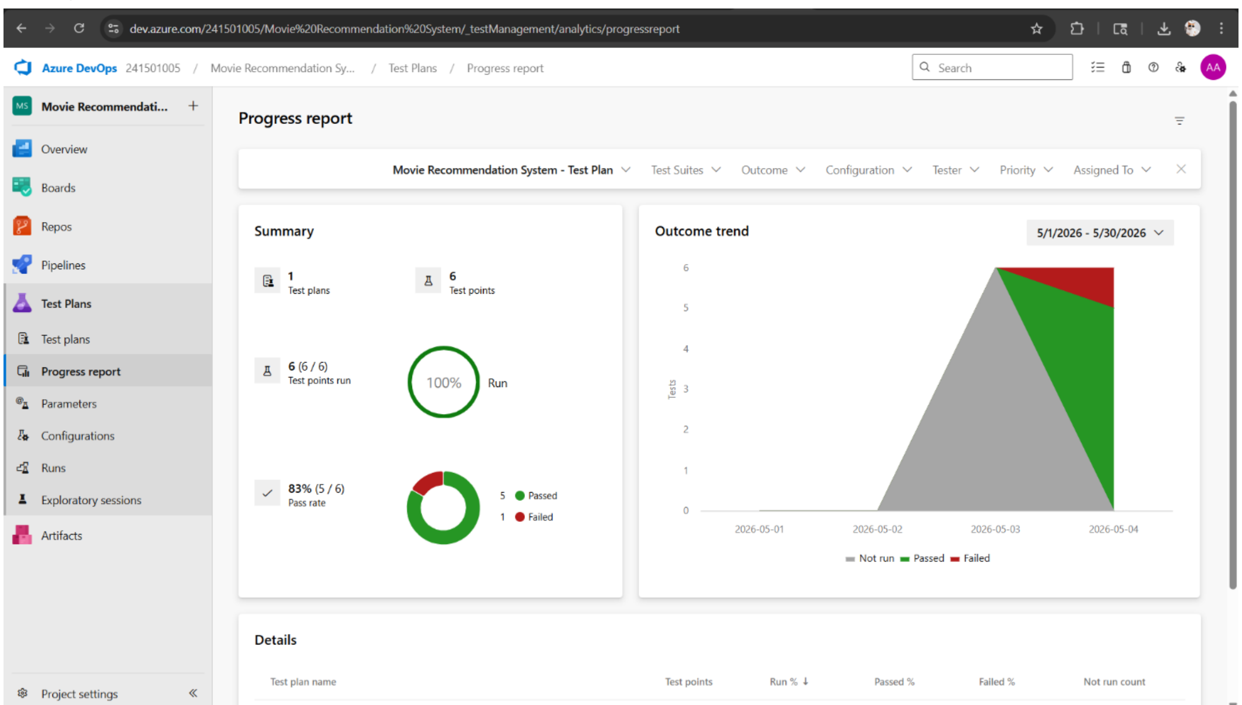
9.Test report summary



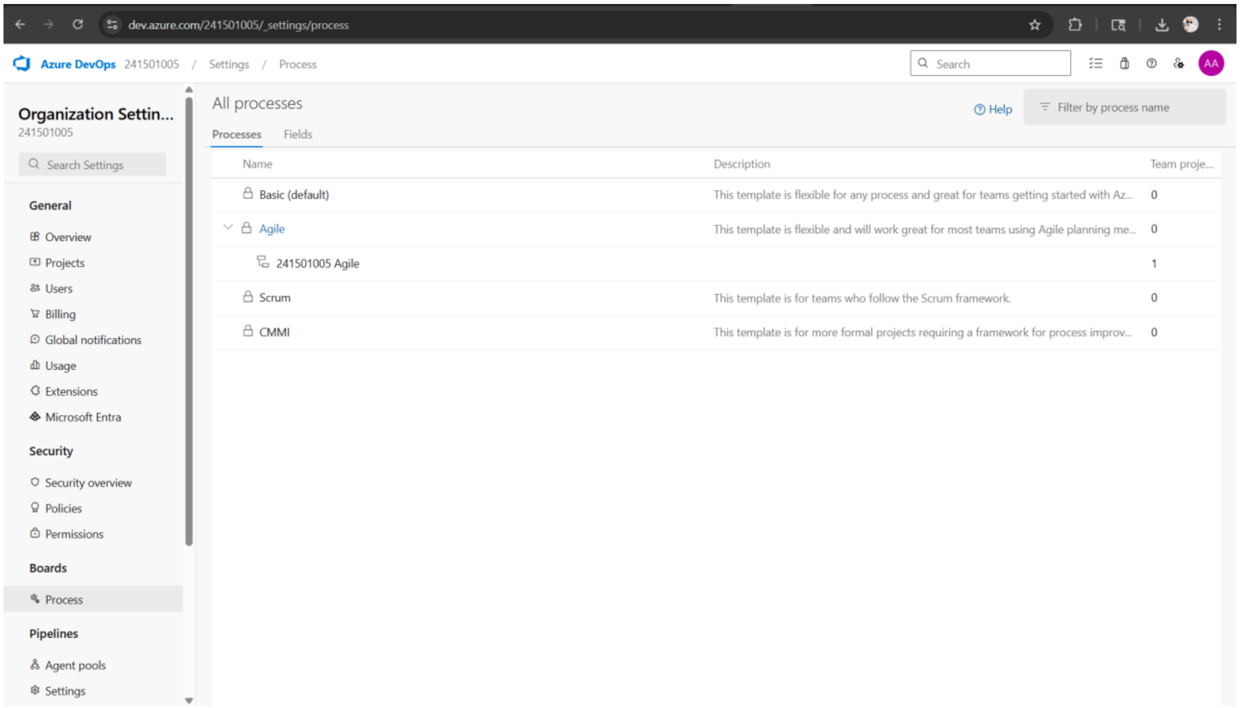
- Assigning bug to the developer and changing state



10.Progress report



11.Changing the test template



Result:

The test plans and test cases for the user stories is created in Azure DevOps with Happy Path and Error Path

EXP NO: 9	LOAD TESTING AND PERFORMANCE TESTING

Aim:

To create an Azure Load Testing resource and run a load test to evaluate the performance of a target endpoint.

Load Testing

Steps to Create an Azure Load Testing Resource:

Before you run your first test, you need to create the Azure Load Testing resource:

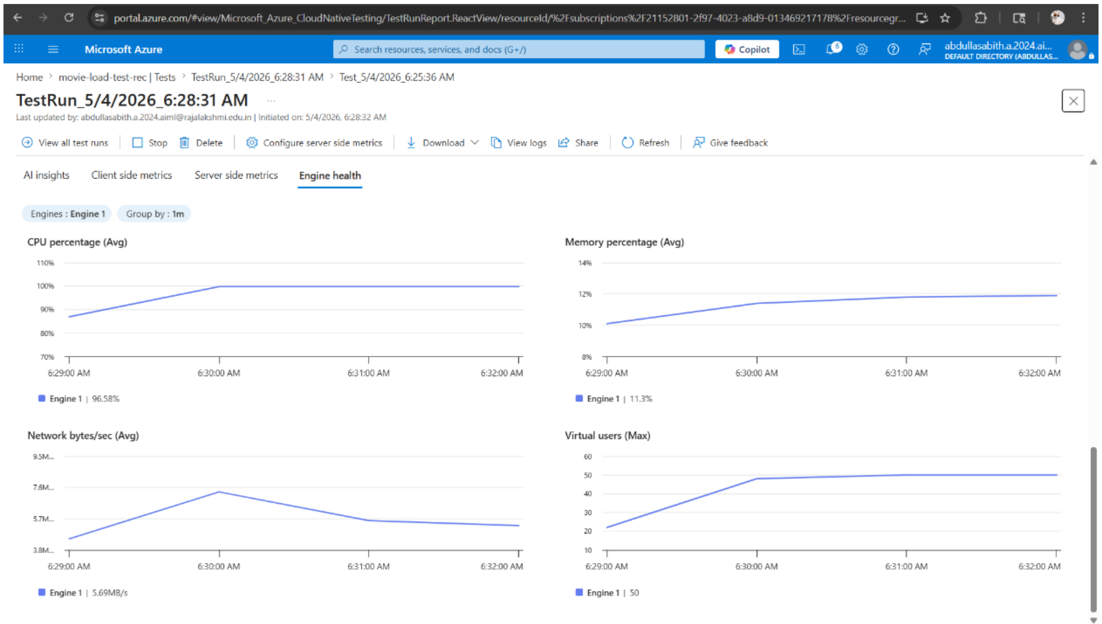
1. Sign in to Azure Portal
   Go to https://portal.azure.com and log in.
2. Create the Resource ○ Go to *Create a resource* → Search for "Azure Load Testing".
   - Select Azure Load Testing and click Create.
3. Fill in the Configuration Details ○ Subscription*: Choose your Azure subscription. ○ *Resource Group:* Create new or select an existing one. ○ *Name:* Provide a unique name (no special characters).
   - *Location:* Choose the region for hosting the resource.
4. (Optional) Configure tags for categorization and billing.
5. Click Review + Create, then Create.
6. Once deployment is complete, click Go to resource.

Steps to Create and Run a Load Test:

Once your resource is ready:

1. Go to your Azure Load Testing resource and click Add HTTP requests > Create.
2. Basics Tab ○ *Test Name:* Provide a unique name.
   - *Description:* (Optional) Add test purpose.
   - *Run After Creation:* Keep checked.
3. Load Settings ○ *Test URL:* Enter the target endpoint (e.g., https://yourapi.com/products).
4. Click Review + Create → Create to start the test.

Load Testing



Result:

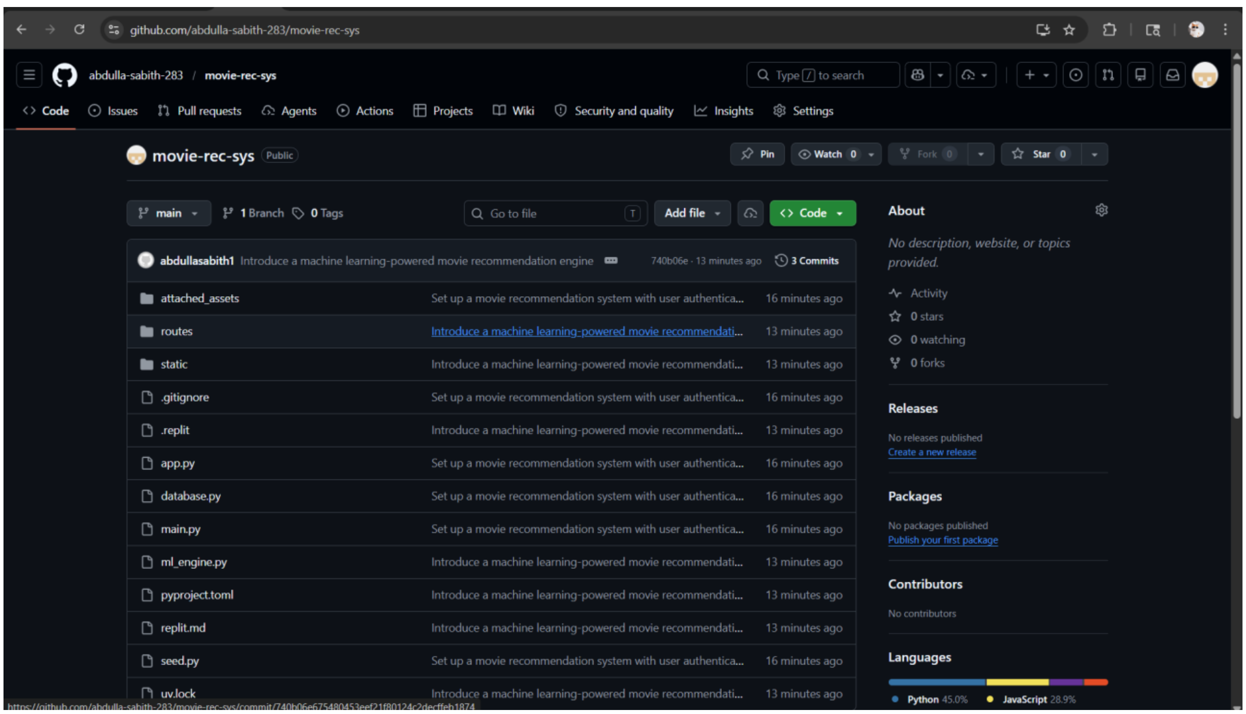
Successfully created the Azure Load Testing resource and executed a load test to assess the performance of the specified endpoint.

EXP NO: 10	**GITHUB: PROJECT STRUCTURE & NAMING CONVENTIONS**

Aim:

To provide a clear and organized view of the project's folder structure and file naming conventions, helping contributors and users easily understand, navigate, and extend the Music Playlist Batch Creator project.

GitHub Project Structure



Result:

The GitHub repository clearly displays the organized project structure and consistent naming conventions, making it easy for users and contributors to understand and navigate the codebase.